European Journal of Operational Research 263 (2017) 461–478

Contents lists available at ScienceDirect

European Journal of Operational Research

journal homepage: www.elsevier.com/locate/ejor

Production, Manufacturing and Logistics

Order picking with multiple pickers and due dates – Simultaneous solution of Order Batching, Batch Assignment and Sequencing, and Picker Routing Problems

André Scholz[a,*], Daniel Schubert[a], Gerhard Wäscher[a,b]

[a] Faculty of Economics and Management, Otto-von-Guericke University Magdeburg, Magdeburg 39106, Germany
[b] School of Mechanical, Electronic and Control Engineering, Beijing Jiaotong University, Beijing 100044, China

ARTICLE INFO

Article history:
Received 22 July 2016
Accepted 20 April 2017
Available online 27 April 2017

Keywords:
Traveling salesman
Order batching
Batch sequencing
Picker routing
Variable neighborhood descent

ABSTRACT

In manual picker-to-part order picking systems, human operators (order pickers) walk or ride through the warehouse, retrieving items from their storage locations in order to satisfy a given demand specified by customer orders. Each customer order is characterized by a certain due date until which all items included in the order are to be retrieved. For the actual picking process, customer orders may be grouped (batched) into more substantial picking orders (batches). The items of a batch are then collected on a picker tour. Thus, the picking process of each customer order in the batch is completed when the picker returns to the depot after the last item of the batch has been picked. Whether and to what extent due dates are violated depends on how the customer orders are batched, how the batches are assigned to order pickers, how the assigned batches are sequenced and how the pickers are routed. Existing literature has only dealt with specific aspects of this problem so far. In this paper, for the first time, an approach is proposed which considers all subproblems simultaneously. A mathematical model of the problem is introduced that allows for solving small problem instances. For larger instances, a variable neighborhood descent algorithm is presented. By means of numerical experiments, it is demonstrated that the algorithm provides solutions of excellent quality. Furthermore, it is shown that a simultaneous solution approach to the above-mentioned subproblems can be considered as a significant source for improving the efficiency of operations in distribution warehouses.

© 2017 Elsevier B.V. All rights reserved.

1. Introduction

Order picking is a function which is critical for managing and operating distribution warehouses efficiently. It deals with the retrieval of items requested by external or internal customers (Petersen & Schmenner, 1999; Wäscher, 2004). In picker-to-part systems, which are referred to in this paper, human operators (order pickers) walk or ride through the warehouse and collect the requested items from their storage locations (Wäscher, 2004).

The items specified by a customer order usually have to be provided until a certain due date (Henn & Schmid, 2013). Violation of due dates may delay subsequent shipment and/or production processes, and, as a consequence, results in an unacceptable customer satisfaction and high costs. Whether or to what extent due dates of a set of customer orders can be met is dependent on (1) how the customer orders are grouped into picking orders (Order Batching Problem), (2) how the picking orders are assigned to and sequenced by the order pickers (Batch Assignment and Sequencing Problem), and (3) how each order picker is routed in order to collect the items of each picking order (Picker Routing Problem). These problems are closely interrelated. Thus, solving them simultaneously appears to be a promising approach for the provision of solutions which comply with the given due dates in the best possible way. Literature dealing with solution approaches which explicitly take into account these problems simultaneously is almost non-existing. To the best of our knowledge, Chen, Cheng, Chen, and Chan (2015) represent the only exception. Their approach is related to a problem environment which is more specific than the one considered in this paper. Furthermore, computing times become a critical issue and the numerical experiments demonstrate that this approach can only be applied to very small problem instances. Due to this reason, the approach of Chen et al. (2015) is neither suitable for dealing with practical-sized problem instances nor can it be used for evaluating the benefit which results from solving the subproblems simultaneously.

* Corresponding author.
E-mail address: andre.scholz@ovgu.de (A. Scholz).

http://dx.doi.org/10.1016/j.ejor.2017.04.038
0377-2217/© 2017 Elsevier B.V. All rights reserved.

Consequently, in this paper, we present a new, more competitive approach to what is called hereafter the Joint Order Batching, Assignment and Sequencing, and Routing Problem (JOBASRP) and which includes an integrative view of the problems sketched above. We propose a mathematical programing formulation to this problem whose size increases polynomially with the number of customer orders. This model provides insights into the problem but is only appropriate for solving small problem instances. Therefore, we also introduce a heuristic solution approach, namely a variable neighborhood descent algorithm, which incorporates neighborhood structures regarding the batching and the sequencing problem proposed in an earlier paper by Henn (2015). The arising routing problems are solved by means of the combined heuristic, which constructs routes of good quality within fractions of a second (Roodbergen & de Koster, 2001a). In order to improve the routes, the Lin–Kernighan–Helsgaun heuristic (Helsgaun, 2000) is applied to very promising solutions. By means of numerical experiments, it is shown that this approach leads to high-quality solutions within reasonable computing times even for large problem instances. Furthermore, a sequential approach is constructed which is composed of state-of-the-art algorithms for the respective subproblems. In the experiments, the sequential and the joint approach are compared with respect to the solution quality in order to provide an insight into the benefits of dealing with the JOBASRP as a holistic problem. It is pointed out that the joint consideration of the subproblems results in a substantial reduction of the tardiness of all customer orders and that the application of an integrated approach is inevitable in order to obtain high-quality solutions.

The remainder of this paper is organized as follows: in Section 2, we give a precise statement of the JOBASRP. Section 3 comprises a literature review regarding the subproblems and joint problems. For the JOBASRP, a new mathematical model formulation is presented in Section 4. Section 5 contains the description of the variable neighborhood descent algorithm including the generation of an initial solution, the different neighborhood structures and the integration of the routing algorithms. In Section 6, the numerical experiments are presented which have been carried out in order to evaluate the performance of the algorithm as well as the benefits resulting from solving the subproblems simultaneously. The paper concludes with an outlook on further research.

2. Problem description

We consider a warehouse with a manual, low-level picker-to-parts order picking system from which a given set of items has to be retrieved. The items are stored on pallets or in bins directly accessible to the order pickers (Henn, Koch, & Wäscher, 2012). The storage locations of the items typically constitute a block layout (Roodbergen, 2001) composed of so-called picking aisles and cross aisles. The picking aisles run parallel to each other and include storage locations arranged on both sides of each picking aisle. Cross aisles do not contain any storage locations but enable order pickers to enter or exit a picking aisle. Furthermore, the cross aisles divide the picking area into several blocks and the picking aisles into subaisles. A block is formed by the picking area located between two adjacent cross aisles. The corresponding part of a picking aisle is denoted as a subaisle. Thus, a warehouse with m picking aisles and $q+1$ cross aisles includes q blocks and $q \cdot m$ subaisles. Additionally, the warehouse contains a depot where the order pickers enter the picking area and return to in order to deposit the picked items. In Fig. 1, an example of a picking area with two blocks and five picking aisles is depicted. A two-block layout is characterized by three cross aisles, namely the front, middle and rear cross aisle, where the front and the rear cross aisles represent the cross aisles nearest to and farthest away from the depot, respectively. The middle cross aisle separates the two blocks from each other. The storage locations are represented by rectangles, while black rectangles symbolize the locations of requested items (pick locations).

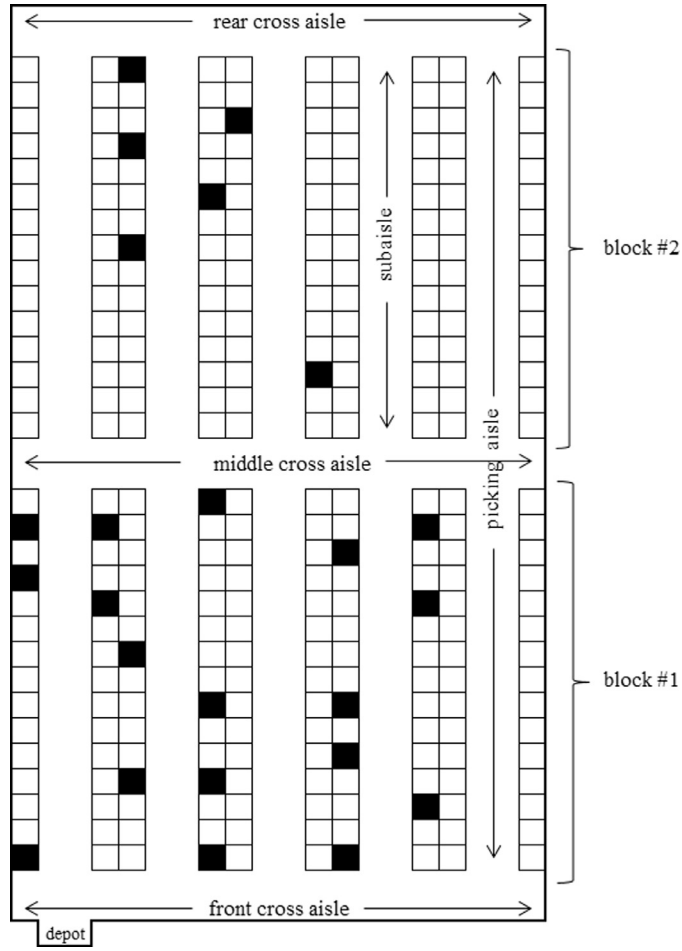

Fig. 1. Two-block layout.

In order to retrieve the requested items, several order pickers operate in the picking area. Each order picker is equipped with a picking device, e.g. a cart or a roll cage, enabling him to perform a tour through the warehouse on which several items are picked before he returns to the depot. The maximum number of stops on a tour is dependent on the capacity of the picking device and the capacity requirements of the respective customer orders. On his tour, the order picker is guided by a so-called pick list. The list represents a batch and identifies the storage locations and the quantities of the items which are to be retrieved on the same tour. A batch may include requested items of several customer orders. However, splitting of customer orders is not allowed since it would result in an unacceptable sorting effort. The pick list also contains information on the sequence according to which a picker is meant to visit the respective pick locations.

The time an order picker spends for retrieving all items of a batch (batch processing time) can be divided into (Tompkins, White, Bozer, & Tanchoco, 2010):

- the setup time, i.e. the time needed for preparing a tour,
- the search time, i.e. the time required at each pick location for identifying the correct item,
- the pick time, i.e. the time for physically retrieving the items from their storage locations, and

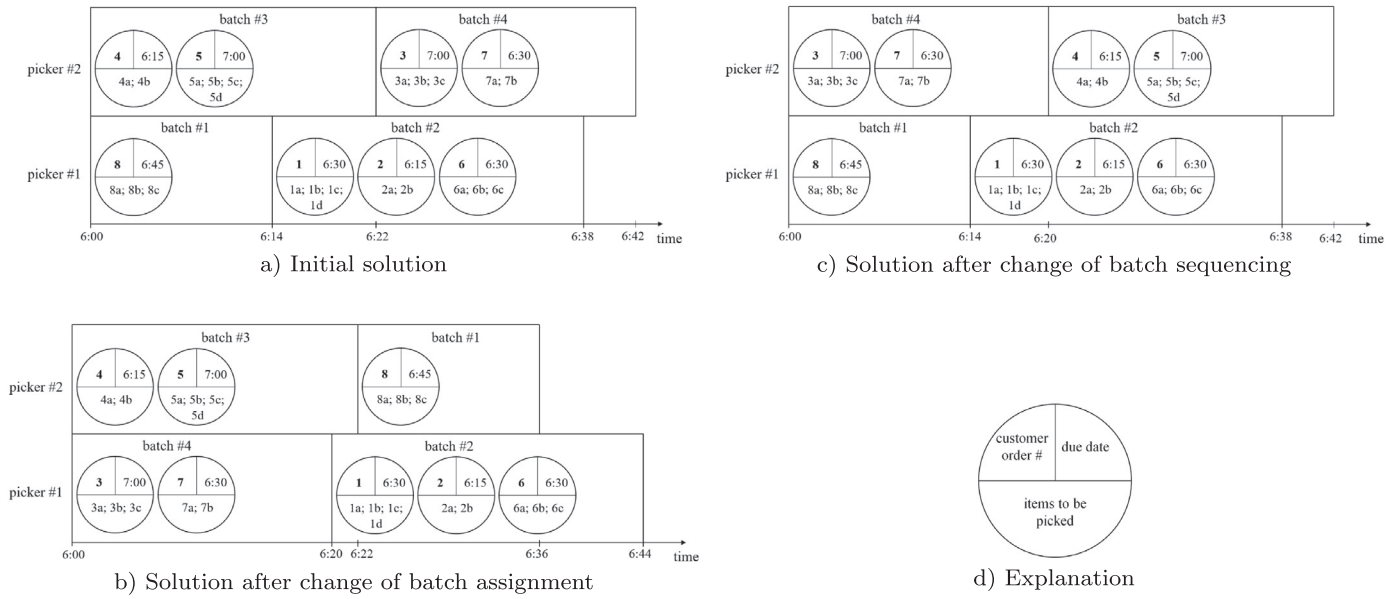

Fig. 2. Impact of batch assignment and sequencing decisions.

- the travel time, i.e. the time spent for traveling from the depot to the first pick location, between the pick locations and from the last pick location back to the depot.

Processing of a batch is started when the corresponding order picker to whom the batch has been assigned starts with preparing the tour. This point in time is addressed as the batch start time, while the batch completion time denotes the point in time when the picker returns to the depot after having retrieved all items included in the batch. The start time (likewise: the completion time) of a customer order, therefore, is identical to the start time (completion time) of the batch in which the order is included. An example including 8 customer orders to be processed by two order pickers is given in Fig. 2. In Fig. 2(a), a Gantt chart is depicted in which each batch is represented by a box. Each batch consists of several customer orders, each of which, again, including several items to be picked, e.g. batch #4 includes order #3 (items to be picked: 3a, 3b, 3c) and order #7 (items to be picked: 7a, 7b). According to this chart (and based on the given definition) the completion times of customer orders #3 and #7 are identical and both orders are completed at 6:42.

Given a particular composition of the batches, a different assignment of batches to order pickers (see Fig. 2(b)) and/or a different sequence of the batches assigned to a picker (see Fig. 2(c)) will result in different order completion times. From Fig. 2(b) and (c) we take, e.g. that the completion time of customer orders #3 and #7 now is 6:20, respectively. Of course, according to Fig. 2(b), also the completion times of the orders included in batch #1 (order #8), in batch #2 (orders #1, #2 and #6), and according to Fig. 2(c), also the completion time of batch #3 (orders #4 and #5) has been affected.

Furthermore, the completion time of customer orders can also be influenced by the assignment of orders to batches. The length of a box, i.e. its extent in the horizontal dimension, indicates the batch processing time. It is determined by the customer orders from which it is composed and by the time the order picker needs to complete a respective tour through the warehouse on which the requested items are collected. In Fig. 3(a), the schedule of Fig. 2(a) is repeated. An example of a tour corresponding to batch #2 is presented in Fig. 3(b). In Fig. 3(c), the batches are composed differently. Batch #2 now includes orders #2, #6 and #7, while order #1 has been assigned to batch #4. The corresponding picker tour through the warehouse related to batch #2, depicted in Fig. 3(d), is different to the one of Fig. 3(b). In particular, since the tour is now shorter, less time will be necessary for its completion, resulting in a decrease of the processing time of batch #2, i.e. a decrease of the length of the box related to batch #2 in Fig. 3(c). Likewise, also the processing time of batch #4 to which order #1 has been assigned will be affected. We note again that also completion times of all customer orders may be affected which are included in batches scheduled after those batches to which such changes occurred.

In distribution warehouses, customer orders usually have to be picked until certain due dates (Henn & Schmid, 2013) in order to guarantee the scheduled departure of trucks delivering the picked items during contracted (and often tight) time windows to external customers (Gademann, van den Berg, & van der Hoff, 2001). As can be taken from the examples presented in Figs. 2 and 3, whether such due dates are met or not depends on the assignment of customer orders to batches, the assignment of batches to order pickers, the sequencing of batches and the routing of order pickers. In the example from Fig. 2, orders #7 and #8 are due at 6:30 and 6:45, respectively. According to the solution depicted in Fig. 2(a), order #7 is completed at 6:42, i.e. it is delayed by 12 minutes, whereas order #8 is completed in time. In Fig. 2(b), the assignment of the batches, which include the two orders, to the order pickers is changed. Order #7 is now completed at 6:20, while the completion time of order #8 is 6:36. Thus, the due dates of both orders are met. However, this affects the completion time of batch #2 which is increased by one minute and, therefore, the extent to which orders #1, #2 and #6 are delayed is also increased.

Delayed shipments result in fines which, e.g. for the delivery of fresh foods to supermarkets or of parts to production lines in the car industry, are often dependent on the length of the delay (tardiness). More precisely, the tardiness τ_n of a customer order n is defined as the (non-negative) difference between the completion time c_n of the order and its due date d_n (Henn & Schmid, 2013), i.e. the tardiness is given by $\tau_n = \max\{c_n - d_n; 0\}$. The sum of the tardiness of all customer orders is referred to as the total tardiness. A total tardiness of 0 means that the due dates of all customer orders are met in the respective solution. The total tardiness of all customer orders will be used here for the evaluation of solutions provided by our approach. We note that this objective

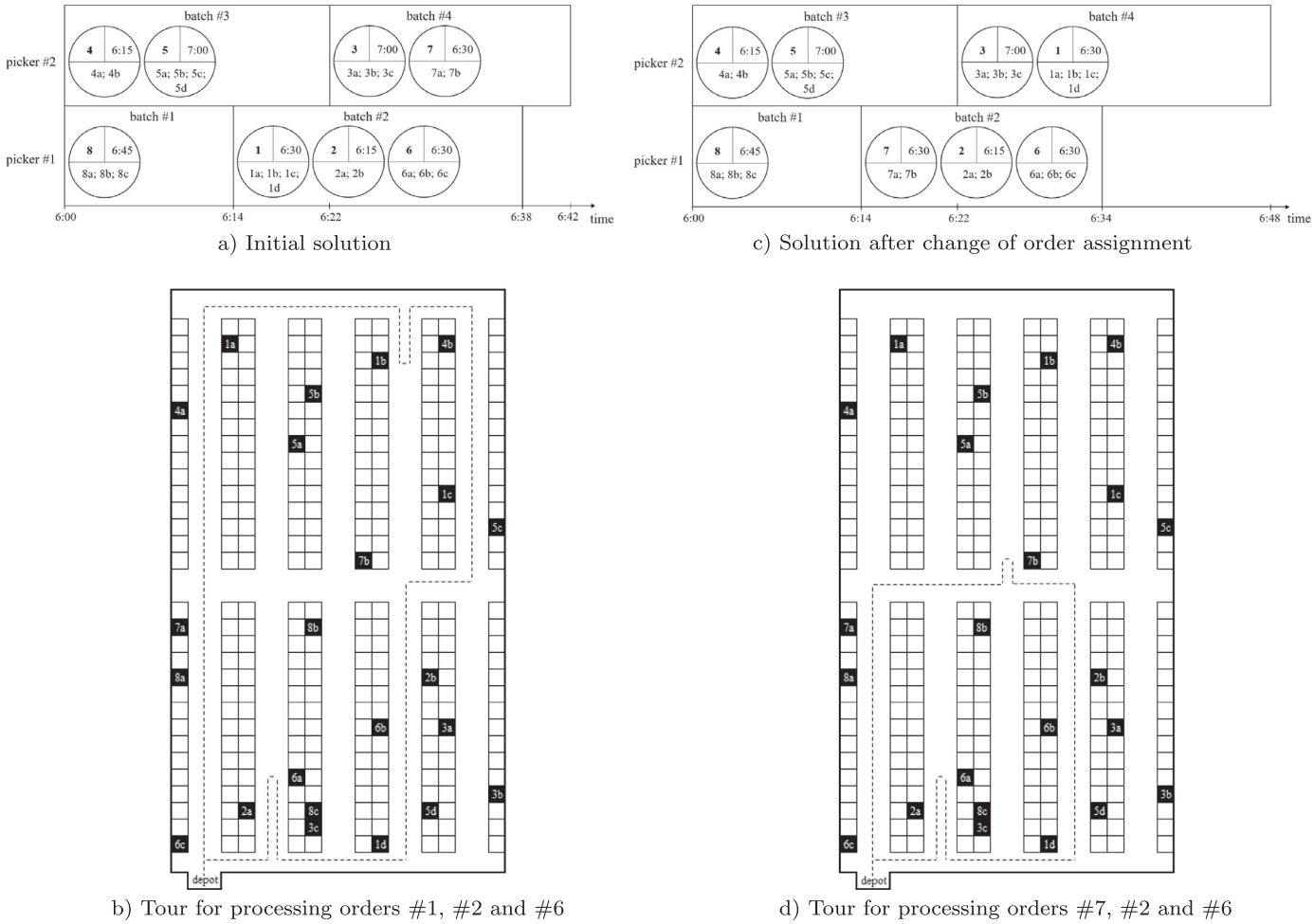

Fig. 3. Impact of order batching decisions on picker routing.

is widely used; not only in the literature related to order picking (Elsayed, Lee, Kim, & Scherer, 1993; Henn & Schmid, 2013; Tsai, Liou, & Huang, 2008) but also in literature related to other problem settings in which on-time delivery to external or internal clients represents a key element of the study, e.g. in scheduling (Koulamas, 1994; Pinedo, 2016; Ullrich, 2013).

The Joint Order Batching, Assignment and Sequencing, and Routing Problem can now be stated as follows: let a non-empty set of customer orders be given, each of which including certain items with known storage locations to be removed from the warehouse. Furthermore, each order is characterized by a due date until which all requested items of the order should be retrieved and brought forward to the depot. A given number of order pickers is available for carrying out the necessary picking operations. Then, the following questions have to be answered (simultaneously) in such a way that the total tardiness is minimized:

- How should the set of customer orders be grouped into picking orders? (Order Batching Problem)
- How and in which sequence should the set of picking orders be assigned to the order pickers? (Batch Assignment and Sequencing Problem)
- For each picking order, in which sequence should the respective pick locations be visited? (Picker Routing Problem)

In the following, we will assume a warehouse with wide aisles which enable the order pickers to pass each other. This assumption has also been made by Chen et al. (2015) and Henn (2015) and allows for neglecting cases in which the processing time of a batch is increased by waiting times which may arise if order pickers simultaneously work in the same picking aisle. Even without considering such picker blocking aspects, the JOBASRP formulated above is known to be NP-hard (Chen et al., 2015).

3. Literature review

Although the above-mentioned subproblems of the JOBASRP arise simultaneously, joint solution approaches have rarely been addressed in the literature so far. Instead, the subproblems are dealt with independently of each other in most approaches. The first subproblem is the Order Batching Problem (OBP) which can be stated as follows (Wäscher, 2004): given the article storage locations, the routing strategy to be used, and the capacity of the picking device, how can the set of customer orders be grouped into picking orders such that the total lengths of the arising tours is minimized? The OBP has been widely studied in the literature and a large variety of solution approaches exists. For a very detailed review of solution methods to the OBP, we refer to de Koster, Le-Duc, and Roodbergen (2007) and Henn et al. (2012).

As for the Batch Assignment and Sequencing Problem (BASP) customer orders are characterized by due dates which have to be met in the best possible way, while each customer order has already been assigned to a certain batch. Solutions are evaluated by the total tardiness of all orders. The BASP can then be stated as follows: how should the batches be assigned to a limited number of pickers and, for each picker, how should the batches be sequenced such that the total

Table 1
Joint considerations regarding the Batching, Assignment and Sequencing, and Routing Problem.

Reference	Batching	Assignment	Sequencing	Routing
Kulak, Sahin, and Taner (2012)	✓			✓
Grosse, Glock, and Ballester-Ripoll (2014)	✓			✓
Scholz and Wäscher (2017)	✓			✓
Elsayed et al. (1993)	✓		✓	
Elsayed and Lee (1996)	✓		✓	
Henn and Schmid (2013)	✓		✓	
Henn (2015)	✓	✓	✓	
Tsai et al. (2008)	✓		✓	✓
Chen et al. (2015)	✓		✓	✓

tardiness is minimized? The special cases of the BASP with a single customer order per batch or with customer orders having identical due dates find their parallels in Parallel Machine Scheduling: a set of jobs (here: batches) has to be assigned to machines (here: order pickers) and sequenced in such a way that the total tardiness of all jobs is minimized (Pinedo, 2016). We refer to Koulamas (1994) for a review of solution approaches to the Parallel Machine Scheduling Problem. To the best of our knowledge, no solution approach exists to the BASP where the batches include customer orders with different due dates.

The Picker Routing Problem (PRP) can be stated as follows (Ratliff & Rosenthal, 1983; Scholz, Henn, Stuhlmann, & Wäscher, 2016): given a set of items to be picked from known storage locations, in which sequence should the locations be visited such that the total length of the corresponding picker tour is minimized? Although the PRP can be solved efficiently in warehouses with up to two blocks (Ratliff & Rosenthal, 1983; Roodbergen & de Koster, 2001b), simple routing strategies are used for routing order pickers in practice. This is due to the fact that optimal tours appear to be complex and difficult to memorize for the order pickers. The S-shape strategy is the most frequently used routing scheme in practice (Roodbergen, 2001). When applying this strategy, the order picker traverses each subaisle completely which contains at least one requested item. An exception may occur in the last subaisle of the block which is visited. Here, the picker may return after retrieving all items in this subaisle. A more sophisticated strategy, the so-called combined strategy (Roodbergen & de Koster, 2001a), combines elements of the S-shape and the return strategy. For each picking aisle to be visited, by means of dynamic programing it is determined whether the aisle is traversed or whether it is entered and left via the same cross aisle. For a detailed review of routing strategies, we refer to de Koster et al. (2007) (single-block layout) and Roodbergen and de Koster (2001a) (multi-block layout). As the PRP is a special case of the Traveling Salesman Problem (TSP), Theys, Bräysy, Dullaert, and Raa (2010) applied a TSP heuristic, namely the Lin–Kernighan–Helsgaun heuristic (Helsgaun, 2000), to the PRP and demonstrated that the tour length can be reduced by up to 48% compared to the usage of the simple S-shape strategy.

In more recent years, research has shifted to solution approaches which focus on the simultaneous solution of two or more subproblems of the JOBASRP. Table 1 provides an overview of respective publications. The table shows that all approaches deal with the OBP and either include the BASP or the PRP. As for the BASP, the assignment and the sequencing problem are treated separately. A check mark provided in the sequencing but not in the assignment column indicates that a single picker is considered only, i.e. no assignment decisions have to be made in this case. In the following part of the review, we focus on those articles dealing with three of the four decision types.

In Henn (2015), the OBP is combined with the BASP, resulting in the Joint Order Batching, Assignment and Sequencing Problem (JOBASP). This is the only paper so far in which multiple pickers have been taken into account. Henn (2015) proposed a variable neighborhood descent (VND) and a variable neighborhood search (VNS) approach in order to solve the JOBASP. As an upper bound, an earliest start date rule (ESDR)-based algorithm has been chosen in which customer orders are batched and sequenced according to their due dates, and batches are assigned to the picker who currently possesses the smallest total processing time. Problem instances with up to 200 customer orders have been included in the numerical experiments. The total tardiness obtained by using the ESDR-based algorithm could be reduced by 41% (VNS) and 39% (VND) on average. Application of the VNS algorithm requires up to 25 minutes of computing time, while the VND approach terminates after a maximum of 30 seconds.

In general, from all the papers discussed so far, it becomes clear that the simultaneous solution of the subproblems of the JOBASRP may provide large benefits w.r.t. improved planning of picking operations. Nevertheless, only two approaches exist which deal with the complete JOBASRP. Tsai et al. (2008) proposed a genetic algorithm for the JOBASRP. Apart from the total tardiness, they also minimize the total earliness as well as the total tour length. Unlike in the approaches discussed before, splitting of customer orders is allowed. Chen et al. (2015) presented a genetic algorithm for the JOBASP and solved the arising PRPs by means of an ant colony approach. In the genetic algorithm, however, the batching and the sequencing problem are considered separately which results in many unfeasible solutions with respect to the capacity constraint of the picking device. Furthermore, the ant colony algorithm consumes far more computing time than problem-specific approaches to the PRP do. Thus, it is not surprising that the authors stated that computing times are a critical issue (without reporting any computing times, though). In their numerical experiments, only very small problem instances with up to 8 orders have been considered. Both the study of Tsai et al. (2008) and Chen et al. (2015) have in common that it is assumed that only a single picker is available for processing the customer orders. This implies that the assignment problem does not have to be taken into account which simplifies the problem considerably.

We conclude that, even though the joint solution of the subproblems of the JOBASRP seems to represent a promising research path, no approach exists which captures the core elements of the problem and, at the same time, is capable of providing solutions of good quality in reasonable computing time.

4. Model formulation

The mathematical model introduced by Henn (2015) for the JOBASP can be adapted to the JOBASRP. It requires that all feasible batches have to be generated in advance. Then, for each feasible batch, the minimum processing time has to be computed, which involves solving the arising PRPs to optimality. As a consequence, when solving a specific problem instance, providing the problem data for the model would already consume a large amount of computing time. Furthermore, the number of variables in the

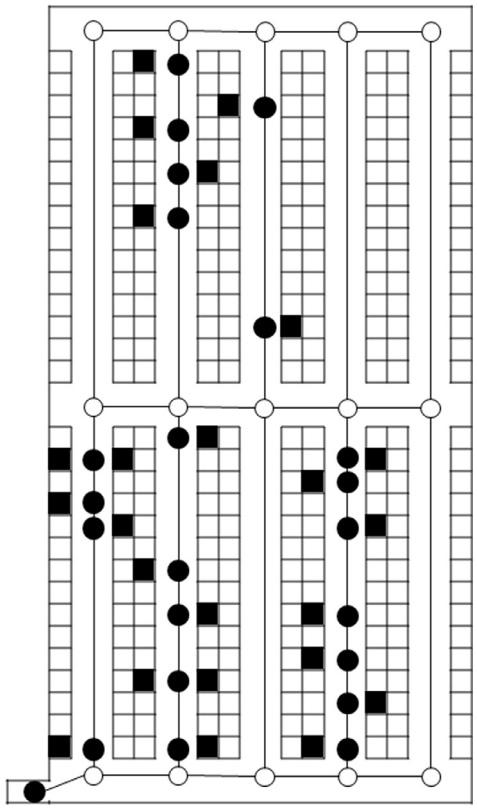

Fig. 4. Illustration of a Steiner TSP.

model depends on the number of feasible batches, which increases exponentially with the number of customer orders. Thus, even for small instances, it may not be possible to generate the model explicitly at all due to memory restrictions.

Instead, in order to keep the number of variables at a reasonable level, we chose a modeling approach in which the batches are not generated in advance. For our model formulation as well as for the heuristic approach presented in Section 5, we assume that the capacity of the picking device is measured in the number of items, i.e. a maximum number of C items can be picked on a tour. This is a standard assumption which has also been made by Bozer and Kile (2008) and Henn (2015). Note, that this is not a critical assumption as the mathematical model and the heuristic approach can easily be modified such that other kinds of capacity constraints (e.g. a maximum number of customer orders or a maximum total weight of items) can be dealt with.

The model formulation can be divided into two parts: the first part is related to the Joint Order Batching and Picker Routing Problem which means that the customer orders are grouped into batches and the corresponding tours are constructed. With respect to the specific structure of tours in an order picking warehouse, we interpret the routing problem as a Steiner TSP. The graph of Fig. 4 illustrates this interpretation and will be used as the basis for our model.

Black vertices represent the location of the depot and the pick locations. These vertices have to be included in the tour. White vertices (Steiner points) depict the intersections of picking aisles and cross aisles. They can be visited, but do not necessarily need to be included in the tour. As can be taken from the figure, the maximum degree of a vertex is equal to four. Therefore, this Steiner TSP-based graph contains significantly fewer arcs than a standard TSP graph would include. In the model formulation, we consider a directed graph which means that each edge is replaced by two reverse arcs.

The second part of the model formulation deals with the BASP in which the constructed batches are assigned to a picker and arranged in a certain sequence. The sequence consists of as many positions as batches have been assigned to the picker. The picker starts processing the batch assigned to the first position and, after having returned to the depot, the batch in the next position is dealt with etc. This procedure provides the completion time for each batch and the completion times of all customer orders included in the batch, from which the tardiness of each customer order can be determined.

Before presenting the model formulation, we introduce the corresponding sets, parameters and variables.

Sets

P:	set of order pickers
K:	set of positions (for each order picker) where a batch can be scheduled $\left(K = \left\{1, \ldots, \overline{K}\right\}\right)$
H:	set of tours which can be performed
N:	set of customer orders
V:	set of vertices in the graph representing the warehouse
R:	set of vertices representing a pick location and the location of the depot
A:	set of arcs in the graph representing the warehouse

Parameters

C:	capacity of the picking device
β^s:	setup time per batch
β^p:	pick and search time per item
β^t:	travel time per length unit
c_n:	number of requested items of order $n \in N$
d_n:	due date of order $n \in N$
α_{nr}:	constant for indicating whether vertex $r \in R$ represents a storage location of a requested item of order $n \in N$ ($\alpha_{nr} = 1$) or not ($\alpha_{nr} = 0$)
w_a:	distance to be covered when using arc $a \in A$
M:	sufficiently large number

$$\left(\text{e.g. } M = \overline{K} \cdot \left(\beta^s + \beta^t \cdot \sum_{a \in A} w_a\right) + \beta^p \cdot \sum_{n \in N} c_n\right)$$

Variables

τ_n:	tardiness of order $n \in N$
$\tilde{u}_h$:	processing time of tour $h \in H$
u_{pk}:	processing time of the tour assigned to position $k \in K$ of picker $p \in P$
v_{pk}:	completion time of the tour assigned to position $k \in K$ of picker $p \in P$
x_{pkh}:	variable for indicating whether tour $h \in H$ is assigned to position $k \in K$ of picker $p \in P$ ($x_{pkh} = 1$) or not ($x_{pkh} = 0$)
y_{nh}:	variable for indicating whether order $n \in N$ is assigned to tour $h \in H$ ($y_{nh} = 1$) or not ($y_{nh} = 0$)
z_{ah}:	variable for indicating whether arc $a \in A$ is included in tour $h \in H$ ($z_{ah} = 1$) or not ($z_{ah} = 0$)
f_{ah}:	number of units of the commodity passing arc $a \in A$ on tour $h \in H$

The JOBASRP can then be formulated as follows.

$$\min \sum_{n \in N} \tau_n \tag{1}$$

$$\sum_{h \in H} x_{pkh} \leq 1 \qquad \forall\, p \in P,\ k \in K \tag{2}$$

$$\sum_{p \in P} \sum_{k \in K} x_{pkh} = 1 \qquad \forall\, h \in H \tag{3}$$

$$\sum_{h \in H} y_{nh} = 1 \qquad \forall n \in N \tag{4}$$

$$\sum_{n \in N} c_n \cdot y_{nh} \leq C \qquad \forall h \in H \tag{5}$$

$$\sum_{a \in \delta_0^-} z_{ah} \geq 1 \qquad \forall h \in H \tag{6}$$

$$\sum_{a \in \delta_r^-} z_{ah} \geq \alpha_{nr} \cdot y_{nh} \qquad \forall h \in H,\ n \in N,\ r \in R \backslash \{0\} \tag{7}$$

$$\sum_{a \in \delta_v^-} z_{ah} = \sum_{a \in \delta_v^+} z_{ah} \qquad \forall h \in H,\ v \in V \tag{8}$$

$$\sum_{a \in \delta_r^+} f_{ah} - \sum_{a \in \delta_r^-} f_{ah} = \alpha_{nr} \cdot y_{nh} \qquad \forall h \in H,\ n \in N,\ r \in R \backslash \{0\} \tag{9}$$

$$\sum_{a \in \delta_v^+} f_{ah} - \sum_{a \in \delta_v^-} f_{ah} = 0 \qquad \forall h \in H,\ v \in V \backslash R \tag{10}$$

$$f_{ah} \leq C \cdot z_{ah} \qquad \forall a \in A,\ h \in H \tag{11}$$

$$\beta^s + \beta^p \cdot \sum_{n \in N} c_n \cdot y_{nh} + \beta^t \cdot \sum_{a \in A} w_a \cdot z_{ah} \leq \tilde{u}_h \qquad \forall h \in H \tag{12}$$

$$\tilde{u}_h - M \cdot \left(1 - x_{pkh}\right) \leq u_{pk} \qquad \forall p \in P,\ k \in K,\ h \in H \tag{13}$$

$$u_{p1} \leq v_{p1} \qquad \forall p \in P \tag{14}$$

$$u_{pk} + v_{p,k-1} \leq v_{pk} \qquad \forall p \in P,\ k \in K \backslash \{1\} \tag{15}$$

$$v_{pk} - d_n - M \cdot \left(2 - x_{pkh} - y_{nh}\right) \leq t_n \qquad \forall n \in N,\ p \in P,\ k \in K,\ h \in H \tag{16}$$

$$x_{pkh},\ y_{nh},\ z_{ah} \in \{0, 1\} \qquad \forall a \in A,\ n \in N,\ p \in P,\ k \in K,\ h \in H \tag{17}$$

$$\tau_n,\ \tilde{u}_h,\ u_{pk},\ v_{pk},\ f_{ah} \geq 0 \qquad \forall a \in A,\ n \in N,\ p \in P,\ k \in K,\ h \in H \tag{18}$$

The objective function (1) minimizes the total tardiness. Constraints (2) ensure that at most one tour is assigned to each position of each picker, while (3) guarantee that each tour is performed. Each customer order has to be processed in exactly one tour which is obtained by meeting restrictions (4). The capacity of the picking device is taken into account by satisfying (5). Constraints (6)–(11) represent the routing constraints. First, constraints (6) ensure that the depot is left on each tour. Here, δ_v^+ (δ_v^-) denotes the set of arcs to which vertex v is an end (start) vertex. Vertex "0" represents the location of the depot. Constraints (7) guarantee that each pick location is visited which corresponds to a requested item of a customer order included in the respective tour. Restrictions (8) are the degree constraints. The following three types of constraints represent subtour elimination constraints. These constraints are related to the single-commodity flow constraints introduced by Letchford, Nasiri, and Theis (2013). Subtours are excluded by ensuring that the picker starts his tour with a certain number of units of a commodity and delivers one unit to each vertex to be visited. In this way, the vertices are enumerated according to their appearance in the tour. In constraints (12), the processing time is determined for each tour, which is composed of the setup time, the time for searching and picking the items and the travel time. The processing time of the tour assigned to a certain position of a certain picker is determined in (13), while constraints (14) and (15) calculate the corresponding completion times. Finally, constraints (16) compute the tardiness for each order. The variable domains are defined in (17) and (18).

The model formulation includes linear constraints only. Furthermore, both the number of variables and the number of constraints increase polynomially with the problem size, which is a major advantage of this model in comparison to the formulations of Chen et al. (2015) and Henn (2015). However, as will be shown later, our model is only suitable for solving relatively small problem instances. For dealing with larger instances, we have developed a variable neighborhood descent approach to the JOBASRP.

5. Variable neighborhood descent

5.1. Overview

Variable neighborhood descent was introduced in Hansen and Mladenović (2001). The general principle of VND consists of exploring the solution space of the problem by means of a sequence of neighborhood structures $\mathcal{N}_1, \ldots, \mathcal{N}_L$. It is started with an incumbent solution s^* and the best neighbor s (in terms of the objective function value) of $\mathcal{N}_1(s^*)$ is determined. If s represents a better solution than s^*, then s becomes the new incumbent solution and the first neighborhood structure $\mathcal{N}_1(s^*)$ is considered again. Otherwise, the exploration of the solution space continues with the next neighborhood. The algorithm terminates if no improvement can be found in the last neighborhood structure $\mathcal{N}_L(s^*)$, i.e. a local optimum is found with respect to all neighborhood structures.

In our VND approach, a solution s is a solution to the JOBASP, i.e. it includes information about how the orders are grouped into batches and in which sequence the batches are to be processed by the pickers. Based on this solution, different PRPs have to be solved in order to determine the corresponding objective function values. Since many PRPs arise during the solution process, we decided to solve them by applying the combined heuristic. This heuristic was particularly designed for warehouses with multiple blocks and outperforms the frequently used S-shape heuristic by far in terms of solution quality (Roodbergen & de Koster, 2001a). The corresponding objective function value is denoted by $f_{\text{Comb}}(s)$.

In contrast to the standard VND procedure, we do not directly return to $\mathcal{N}_1$ after an improvement has been found. Instead, before doing so, we determine a local optimum regarding the current neighborhood structure. The objective function value, the total tardiness, is strongly dependent on the processing times of the batches. In order to reduce processing times, each time when a local optimum has been found, the arising PRPs are solved by means of the Lin–Kernighan–Helsgaun (LKH) heuristic. Applied to the PRP, this heuristic results in very short tours (Theys et al., 2010) and, as a consequence, in shorter processing times. However, since the computational effort is much higher than the effort for the application of the combined heuristic, it is not possible to use the LKH heuristic for evaluating all solutions to be considered in the exploration of the neighborhood structures. Due to this fact, we try to ensure that the LKH heuristic is only applied to very promising solutions. Therefore, we decided to determine a local optimum before returning to $\mathcal{N}_1$. $f_{\text{LKH}}(s)$ denotes the total tardiness of a solution s whose PRPs have been solved by means of the LKH heuristic. A pseudocode of our VND approach is depicted below, while

the generation of the initial solution as well as the neighborhood structures will be dealt with in the next sections.

5.2. Initial solution

For the generation of an initial solution, two constructive approaches are first applied, providing one solution each. The solution with the smaller objective function value is then taken as the initial solution for the VND. The first constructive approach is based on the earliest start date rule and was also used by Henn (2015). It is a priority rule-based algorithm in which orders are assigned successively to the batches and the positions of the pickers. A detailed pseudocode of this algorithm is depicted below.

In the pseudocode, U denotes the set of orders not yet assigned to a picker, k_p is the sequence position of picker p to which the next batch can be assigned and B_{pk_p} represents the set of orders included in the batch assigned to position k_p of picker p. C_p and ν_p denote the number of items contained in the batch under consideration for picker p as well as its completion time, while u_{Comb} calculates the processing time of the current batch by means of the combined heuristic. At the beginning of the algorithm, all orders are unassigned. The orders are then assigned successively to certain batches, starting with the unassigned order n^* with the smallest due date. For each order picker p, the completion time $\tilde{\nu}_p$ is determined that would follow from an assignment of n^* to the picker. The assignment consists of an addition of n^* to batch B_{pk_p} if the capacity constraint of the picking device is not violated; otherwise, the assignment includes the opening of a new batch containing n^*. The algorithm terminates when all orders have been assigned to batches and positions.

In order to minimize the total tardiness, the orders are sorted according to their due dates and then assigned successively. The composition of the batches is not considered any further. This is a reasonable approach as long as the due dates are loose. However, in case of tight due dates, it is of prime importance to construct batches and tours which allow for short processing times. We, therefore, propose another constructive approach which takes into account the processing times resulting from the construction of the batches. This approach can be perceived as a seed algorithm. Seed algorithms have been frequently applied to the OBP and consist of two steps (Elsayed, 1981). First, a seed order is chosen by means of a seed selection rule and assigned to a new batch. According to an order addition rule, orders are then added to this batch. As for the seed selection, we chose an order which is not yet assigned to a batch and has the closest due date. Dependent on the savings in terms of total tardiness, which result from adding an order to this batch instead of processing it separately, other orders are assigned to the batch. A detailed pseudocode of this algorithm can be seen in Algorithm 3.

In step 1, each order is assigned to a separate batch. Starting with the batch that includes the order with the smallest due date, the batches are successively added to the picker p who currently possesses the shortest completion time $\tilde{\nu}_p$. In step 2, batches are merged in order to reduce the processing times as well as the total tardiness. The order with the smallest due date represents the seed order and forms batch $\tilde{B}_1$. Based on the solution generated in step 1, for each unassigned order $n \in U$, savings sav_{in} are determined which are defined as the reduction of the total tardiness obtained by merging the batch of order n and the batch i of the seed order. Of the potential pairs of batches which could be merged without violating the capacity constraint, two batches are actually merged for which the savings are maximal. Then, the savings are updated and another order may be added to the batch. This is done until no further positive savings can be realized. While at least one order exists not considered in step 2, i. e. while $U \neq \emptyset$, the next seed order is determined based on the due dates and a new batch is opened to which orders are to be added. Thus, step 2 provides a solution in which the processing times and the total tardiness are explicitly taken into account. In step 3, the batches in this solution are reassigned to the positions of the pickers. This is done consecutively starting with batch $\tilde{B}_1$. Each batch is inserted into the position k of picker p minimizing the total tardiness $f^{p,k}_{\text{Comb}}$ of all batches which have been inserted up to this point. In analogy to the ESDR-based algorithm, the combined heuristic is used to determine the processing times.

Algorithm 1 Variable neighborhood descent algorithm for the JOBASRP.

Input: problem data, number of neighborhood structures L

Output: solution s^* to the JOBASRP and corresponding total tardiness $f_{\text{LKH}}(s^*)$

```
generate initial solution s;
s* := s;  l := 1;
while l ≤ L do
    s := s*;
    s' := arg min{f_Comb(s̃) | s̃ ∈ N_l(s)};
    while f_Comb(s') < f_Comb(s) do
        s := s';
        s' := arg min{f_Comb(s̃) | s̃ ∈ N_l(s)};
    end while
    if f_LKH(s) < f_LKH(s*) then
        s* := s;
        l := 1;
    else
        l := l + 1;
    end if
end while
```

Algorithm 2 ESDR-based algorithm.

Input: set of orders N with due dates d_n and number of requested items c_n ($n \in N$), set of pickers P, capacity C of the picking device

Output: solution s^* to the JOBASRP and corresponding total tardiness $f_{\text{Comb}}(s^*)$

```
U := N;
for p ∈ P do
    k_p := 1;  B_{pk_p} := ∅;  C_p := 0;  ν_p := 0;
end for
while U ≠ ∅ do
    n* = arg min{d_n | n ∈ U};
    for p ∈ P do
        if C_p + c_{n*} ≤ C then
            ν̃_p := ν_p + u_Comb(B_{pk_p} ∪ {n*});
        else
            ν̃_p := ν_p + u_Comb({n*});
        end if
    end for
    p* := arg min{ν̃_p | p ∈ P};
    U := U \ {n*};  ν_{p*} := ν̃_{p*};
    if C_{p*} + c_{n*} ≤ C then
        B_{p*k_{p*}} := B_{p*k_{p*}} ∪ {n*};  C_{p*} := C_{p*} + c_{n*};
    else
        k_{p*} := k_{p*} + 1;  B_{p*k_p} := {n*};  C_{p*} := c_{n*};
    end if
end while
```

Algorithm 3 Seed algorithm.

Input: set of orders N with due dates d_n and number of requested items c_n $(n \in N)$, set of pickers P, capacity C of the picking device

Output: solution s^* to the JOBASRP and corresponding total tardiness $f_{\text{Comb}}(s^*)$

$U := N$; //step 1
for $p \in P$ **do**
  $k_p := 1$; $v_p := 0$;
end for
while $U \neq \emptyset$ **do**
  $n^* := \arg\min\{d_n \mid n \in U\}$;
  for $p \in P$ **do**
    $\tilde{v}_p := v_p + u_{\text{Comb}}(\{n^*\})$;
  end for
  $p^* := \arg\min\{\tilde{v}_p \mid p \in P\}$;
  $U := U \setminus \{n^*\}$; $v_{p^*} := \tilde{v}_{p^*}$; $k_{p^*} := k_{p^*} + 1$; $B_{p^* k_p} := \{n^*\}$;
end while
$U := N$; $i := 1$; //step 2
while $U \neq \emptyset$ **do**
  $n^* := \arg\min\{d_n \mid n \in U\}$; $\tilde{B}_i := \{n^*\}$; $\tilde{C} := c_{n^*}$;
  $U := U \setminus \{n^*\}$;
  while $\max\left\{sav_{in} \mid n \in U : \tilde{C} + c_n \leq C\right\} > 0$ **do**
    $n^* := \arg\max\left\{sav_{in} \mid n \in U : \tilde{C} + c_n \leq C\right\}$;
    $\tilde{B}_i := B_i \cup \{n^*\}$; $\tilde{C} := \tilde{C} + c_n$; $U := U \setminus \{n^*\}$;
  end while
  $i := i + 1$; $\tilde{C} := 0$;
end while
for $p \in P$ **do** //step 3
  $k_p := 1$;
end for
for $j := 1$ to i **do**
  $(p^*, k^*) := \arg\min\left\{f^{p,k}_{\text{Comb}} \mid p \in P, k \in \{1, \ldots k_p\}\right\}$;
  $B_{p^*,k^*+1} := B_{p^*,k^*}$; ...; $B_{p^*,k_p+1} := B_{p^*,k^p}$; $B_{p^*,k^*} := \tilde{B}_i$;
  $k_{p^*} := k_{p^*} + 1$;
end for

Since the seed algorithm also considers the composition of the batches instead of just basing the construction of a solution on the information about the due dates, it can be expected to lead to better results in case of tight due dates. By selecting the best solution of the ESDR-based and the seed algorithm, we aim to generate an initial solution enabling the VND to proceed faster to a local optimum.

5.3. Neighborhood structures

As mentioned before, a solution to the JOBASRP includes information about the composition of the batches and their assignment to the positions of the pickers. The arising routing problems are only solved in order to determine the resulting objective function value. Thus, the neighborhood structures considered in our VND can be divided into structures related to the sequencing problem and structures regarding the batching problem.

Neighborhood structures related to the sequencing problem only deal with the assignment of the batches to the pickers' positions. The composition of the batches remains unchanged. Thus, complete batches are considered instead of single customer orders. Since the number of batches is usually much smaller than the number of customer orders, these neighborhoods can be explored in reasonable computing time. Therefore, these neighborhoods are used at the beginning of the VND algorithm. As proposed by Henn (2015), we use one sequencing related neighborhood structure $\mathcal{N}_1$.

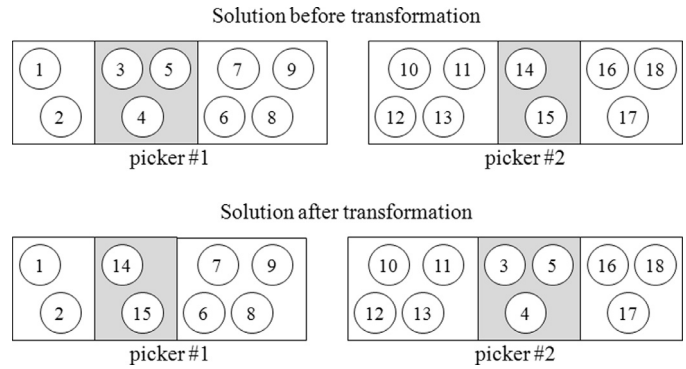

Fig. 5. Example of neighborhood structure $\mathcal{N}_1$.

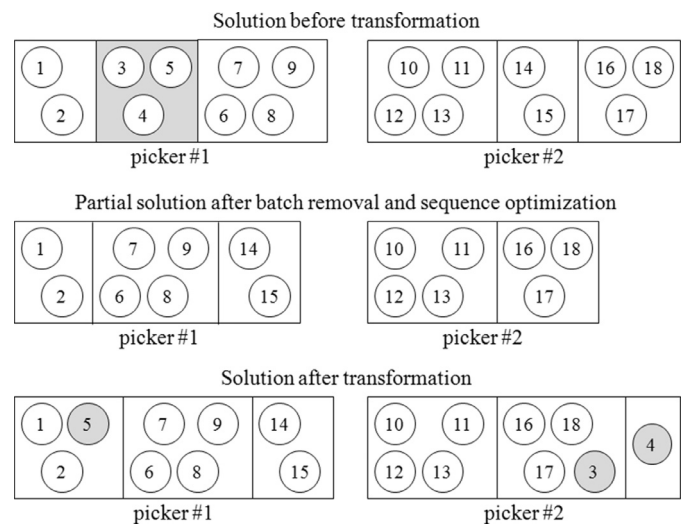

Fig. 6. Example of neighborhood structure $\mathcal{N}_2$.

Regarding $\mathcal{N}_1$, the neighborhood of a solution s is composed of all solutions which can be obtained by exchanging two batches b_1 and b_2 from s. Only exchanges between different pickers are taken into consideration. Batch b_1 is moved to the position of b_2 and vice versa. The positions of the remaining batches are not changed (see Fig. 5).

In $\mathcal{N}_2$, we consider complete batches, too. However, not only the way how batches are sequenced is changed but also how they are composed of orders. This neighborhood structure is meant to break up a complete batch and reassign the orders to other batches. Furthermore, the batch sequence is optimized. A neighbor regarding $\mathcal{N}_2$ is obtained as follows: in a first step, a batch is removed from the solution. In a second step, the remaining sequence is optimized. First, for each batch, it is checked whether the solution can be improved by moving this batch to a position of another picker. The move which provides the largest improvement is carried out. This procedure is repeated until no further improvement is possible. Movements regarding $\mathcal{N}_1$ are then applied as long as the solution can be improved. Subsequent to this improvement step, the orders contained in the removed batch are considered successively in the order of non-descending due dates. Each order may either be inserted in an existing batch or forms a new batch. The option resulting in the smallest total tardiness is realized. An example of a move regarding $\mathcal{N}_2$ is depicted in Fig. 6.

Neighborhood structures $\mathcal{N}_3$ to $\mathcal{N}_6$ are straightforward structures to the batching problem. By means of these structures, the composition of the batches is changed. Either an order is moved from one batch to another batch (shift) or two orders contained in different batches are exchanged (swap). These structures can define movements on the same picker as well as operations including two pickers. Combination of these characteristics (swap or shift and one picker or two pickers) gives rise to four different neighborhood structures which have been used in the following sequence:

$\mathcal{N}_3$: an order is moved from one batch to another batch assigned to the same picker;

$\mathcal{N}_4$: an order is moved from one batch to another batch assigned to another picker;

$\mathcal{N}_5$: two orders are exchanged which are included in different batches assigned to the same picker;

$\mathcal{N}_6$: two orders are exchanged which are included in different batches assigned to different pickers.

We note that only neighbors are considered which represent feasible solutions, i.e. which do not violate the capacity constraint of the picking device. If an order cannot be added to a batch, this order is assigned to a new batch, where the position of the new batch is chosen such that the total tardiness is minimal.

6. Numerical experiments

6.1. Test problem instances

Chen et al. (2015) dealt with a problem which is almost identical to the one discussed here. Unfortunately, the problem instances from their experiments were not available. Therefore, we generated instances with the same characteristics and noticed that all of them could be solved to optimality within a few seconds by means of the model formulation proposed in Section 4. This can be explained by the fact (i) that the problem instances used by Chen et al. (2015) are rather small ones which only included at most 8 customer orders and (ii) that the problem itself is more simple since the authors assumed that only one order picker is available for processing the orders from which follows that no decisions concerning the assignment of batches to pickers have to be made. We, therefore, decided not to use these instances for the evaluation of the solution quality of our VND but design our own, more challenging test problem sets instead.

For our experiments, we adapted the generation of test problems from the numerical experiments of Henn (2015) who dealt with very large instances for the JOBASP. We consider a warehouse with 10 picking aisles, where the number of blocks q varies between 1 and 3, resulting in 10, 20 or 30 subaisles. Each subaisle contains 50 storage locations (25 on each side of the subaisle). The depot is located in front of the leftmost picking aisle, and the distance between the depot and the leftmost picking aisle amounts to 1.5 length units (LUs). For entering or leaving a subaisle, the order picker has to cover a distance of 1 LU. This is also the distance between two adjacent storage locations of a subaisle. The distance between two adjacent picking aisles amounts to 5 LUs.

For the assignment of articles to storage locations, a class-based storage assignment policy is applied, i.e. articles with high demand frequencies are assigned to storage locations near the depot. We use the same approach as Henn (2015), who divided the articles into three classes A, B and C, whereupon class A includes 10% of all articles with the highest demand frequency, representing up to 52% of the total demand; class B contains 30% of all articles responsible for 36% of the demand. The remaining articles, assigned to class C, are characterized by rather low demand frequencies. Within each class, articles are randomly assigned to the storage locations of the corresponding subaisles. The determination of the subaisles is based on the distance to the depot. Class A articles are assigned to storage locations in a subaisle representing 10% of all subaisles with the shortest distance to the depot. The subaisles which belong to 60% of all subaisles farthest from the depot include class C articles.

The number of customer orders is fixed to 100 and 200, whereupon the number of items contained in an order is uniformly distributed over the set $\{5, 6, \ldots, 25\}$. Each order should be completed until a certain due date. The due dates are generated based on the processing times $\tilde{u}_n$ $(n \in N)$ of the orders, the number of available order pickers $p_{\max}$ and the so-called modified traffic congestion rate (MTCR) γ describing the tightness of the due dates (Elsayed & Lee, 1996). The interval from which due dates are randomly chosen is determined as follows: $[\min\{\tilde{u}_n \mid n \in N\}, (2 \cdot (1-\gamma) \cdot \sum_{n \in N} \tilde{u}_n + \min\{\tilde{u}_n \mid n \in N\})/p_{\max}]$. As can be seen, the tightness of the due dates is dependent on the processing time $\tilde{u}_n$ of an order n, while the processing time is determined by the sequence according to which the corresponding items are to be retrieved, which is the main reason why we do not use the same instances as Henn (2015). They applied the simple S-shape and largest gap strategies for the determination of the processing times. Since we integrate the routing problem into our approach, we generate much shorter tours. This results in considerable shorter batch processing times, which is why an application of our approach to the instances of Henn (2015) results in a total tardiness of 0 for most instances. Therefore, we decided to use the LKH heuristic for the determination of the processing times $\tilde{u}_n$ $(n \in N)$ instead, which is more appropriate as the LKH heuristic is also applied within our VND algorithm. The MTCR γ has been fixed to 0.6, 0.7 and 0.8 as done by Henn (2015).

In our experiments, 2, 3 or 5 pickers are available for processing the orders. The capacity C of the picking device is set to 45 or 75 items. The time a picker needs for performing a tour is composed of the setup time, the search and pick time, and the travel time (see Section 2). The setup time amounts to 3 minutes, while searching and picking an item requires 20 seconds, and the order picker moves 20 LU per minute.

Combination of the above-mentioned parameter values results in 108 problem classes. For each problem class, 30 instances have been generated, i.e. 3240 problem instances have been solved in total. The experiments have been carried out on a desktop PC with a 3.4 gigahertz Pentium processor and 8 gigabytes RAM. The solution approach has been encoded in C++ using Microsoft Visual Studio 2015.

6.2. Evaluation of the components of the VND approach

6.2.1. Initial solution

Pretests have shown that the amount of computing time required for applying the VND is strongly dependent on the quality of the initial solution. Starting with a low-quality solution, a large number of iterations has to be carried out until a local optimum is found. Henn (2015) used the ESDR-based algorithm in order to generate an initial solution. However, this rule may result in very poor solutions since the orders are grouped into batches without considering the resulting processing times. Therefore, we proposed a seed algorithm in which both the due dates of the orders and the tour length of the corresponding batches are taken into consideration.

The computing times of both approaches are below one second if the combined heuristic is used for determining the processing times of the respective tours. Since such computing times can be neglected, both approaches are applied and the solution with the smaller total tardiness is used as the initial solution. The impact on the quality of the initial solution is depicted in Tables 2 and 3.

In Tables 2 and 3, for the problem classes with 100 and 200 orders, the quality of the initial solutions (in terms of the aver-

Table 2
Evaluation of the initial solution for 100 orders.

			2 pickers			3 pickers			5 pickers		
C	γ	q	**ESD**	**INI**		**ESD**	**INI**		**ESD**	**INI**	
			tar	tar	imp	tar	tar	imp	tar	tar	imp
45	0.6	1	130	130	0.0	112	112	0.0	89	89	0.0
45	0.6	2	70	70	0.0	165	158	3.7	128	128	0.0
45	0.6	3	235	220	6.3	347	283	18.4	517	516	0.2
45	0.7	1	1831	1017	44.4	1451	1102	24.0	1209	986	18.4
45	0.7	2	2479	1345	45.8	2245	1708	23.9	1483	1280	13.7
45	0.7	3	5347	2715	49.2	3908	2435	37.7	2808	2147	23.5
45	0.8	1	8117	4988	38.6	6181	3996	35.4	4729	3294	30.3
45	0.8	2	10,468	6522	37.7	7203	4734	34.3	6692	4502	32.7
45	0.8	3	14,434	8968	37.9	10,246	6677	34.8	3857	2694	30.2
75	0.6	1	62	62	0.0	85	85	0.0	116	116	0.0
75	0.6	2	63	63	0.0	99	99	0.0	130	130	0.0
75	0.6	3	91	91	0.0	154	154	0.0	270	270	0.0
75	0.7	1	141	141	0.0	166	166	0.0	268	268	0.0
75	0.7	2	181	181	0.0	340	340	0.0	312	312	0.0
75	0.7	3	431	431	0.0	489	489	0.0	686	686	0.0
75	0.8	1	2793	2473	11.5	2556	2399	6.1	2397	2250	6.1
75	0.8	2	3872	3407	12.0	2968	2742	7.6	3726	3134	15.9
75	0.8	3	6477	5120	21.0	5055	4289	15.1	1926	1778	7.7
Average			3179	2108	16.9	2432	1776	13.4	1741	1366	9.9

Table 3
Evaluation of the initial solution for 200 orders.

			2 pickers			3 pickers			5 pickers		
C	γ	q	**ESD**	**INI**		**ESD**	**INI**		**ESD**	**INI**	
			tar	tar	imp	tar	tar	imp	tar	tar	imp
45	0.6	1	158	158	0.0	181	180	0.7	180	180	0.0
45	0.6	2	130	130	0.0	115	115	0.0	270	270	0.0
45	0.6	3	753	604	19.8	345	345	0.0	419	419	0.0
45	0.7	1	6436	2755	57.2	5127	3551	30.7	3187	2784	12.7
45	0.7	2	8150	3461	57.5	5564	4212	24.3	3625	3307	8.8
45	0.7	3	18,697	6972	62.7	12,763	7692	39.7	9461	6865	27.4
45	0.8	1	33,890	19,487	42.5	22,982	14,169	38.3	14,632	9494	35.1
45	0.8	2	41,291	24,360	41.0	26,747	16,759	37.3	17,909	11,872	33.7
45	0.8	3	56,178	32,847	41.5	39,085	23,840	39.0	24,405	15,708	35.6
75	0.6	1	70	70	0.0	85	85	0.0	136	136	0.0
75	0.6	2	70	70	0.0	97	97	0.0	164	164	0.0
75	0.6	3	95	95	0.0	158	158	0.0	267	267	0.0
75	0.7	1	177	177	0.0	190	190	0.0	267	267	0.0
75	0.7	2	375	375	0.0	235	235	0.0	286	286	0.0
75	0.7	3	478	478	0.0	618	618	0.0	755	755	0.0
75	0.8	1	11,039	8705	21.1	7982	7454	6.6	5785	5526	4.5
75	0.8	2	14,669	11,758	19.8	9334	8852	5.2	7471	7011	6.2
75	0.8	3	22,468	16,607	26.1	16,942	13,767	18.7	11,703	9974	14.8
Average			11,951	7173	21.6	8253	5684	13.4	5607	4183	9.9

age total tardiness (tar) in minutes) is depicted which is obtained after the application of the ESDR-based algorithm (ESD) and after the additional application of the seed algorithm (INI). The improvement (imp) [in %] amounts to zero, if the additional application of the seed algorithm has no impact on the quality of the initial solutions. In these cases, solutions constructed by means of the ESDR-based algorithm always lead to a smaller total tardiness for the instances of the corresponding problem class. In fact, this is the case for instances with a very small MTCR ($\gamma = 0.6$) and with a medium MTCR ($\gamma = 0.7$) and a large capacity of the picking device ($C = 75$). These instances are characterized by quite loose due dates which can be met easily. (Note that the generation of the due dates is independent of the capacity of the picking device and due dates can be satisfied easier when the capacity is large since the total processing time of all orders decreases.) If the due dates are not tight, minimizing the processing times gets much less important since the due dates of most orders can be met by processing the orders in the sequence of non-descending due dates. This is exactly what the ESDR-based algorithm guarantees, which is the reason why the application of this rule leads to rather good solutions in these cases.

With an increasing MTCR, the due dates get tighter and harder to meet, resulting in a dramatic increase of the average total tardiness. In case of very tight due dates, it is not sufficient to find a reasonable sequence according to which the orders are to be processed. Instead, minimizing the processing times gets pivotal.

Table 4
Impact of the neighborhood structures for the case of five pickers and 200 orders.

C	γ	$\mathcal{N}_1$	$\mathcal{N}_2$	$\mathcal{N}_3$	$\mathcal{N}_4$	$\mathcal{N}_5$	$\mathcal{N}_6$
45	0.6	25.9	37.8	6.0	11.6	0.9	17.8
45	0.7	42.9	5.8	28.3	8.7	6.3	8.1
45	0.8	33.4	2.1	7.0	9.5	35.1	12.9
75	0.6	3.9	76.0	4.1	6.4	0.2	9.3
75	0.7	12.9	36.8	8.6	18.2	0.9	22.5
75	0.8	26.2	11.1	22.4	12.9	14.8	12.6
Average		24.2	28.3	12.7	11.2	9.7	13.9

Since the processing times are not taken into consideration, solutions obtained by means of the ESDR-based algorithm are expected to be of very poor quality. The results from the numerical experiments confirm this expectation and demonstrate that the seed algorithm outperforms the ESDR-based algorithm by far when a high MTCR is assumed. By means of the seed algorithm, the average total tardiness can be decreased by up to 62.7% (200 orders, 2 pickers, $C = 45,\ \gamma = 0.7,\ q = 3$). As expected, the savings obtained by application of the seed algorithm tend to get larger with a decreasing capacity of the picking device or an increasing MTCR. For problem classes with a small capacity ($C = 45$) and a large MTCR ($\gamma = 0.8$), the reduction of the average total tardiness ranges between 30.2% (100 orders, 5 pickers, $q = 3$) and 42.5% (200 orders, 2 pickers, $q = 1$). Furthermore, it should be noted that the improvements get smaller with an increasing number of order pickers available. This can be deduced to the fact that dealing with the sequencing problem gets more important when a larger number of pickers, i.e. more possibilities of assigning orders, have to be taken into account.

As an intermediate summary, it can be stated that the application of two constructive approaches leads to significant improvements with respect to the total tardiness. On average, across all problem classes the total tardiness can be reduced by 14.2% without a noticeable increase of the computing time.

6.2.2. Neighborhood structures

When designing a VND, another very important issue refers to the choice of the neighborhood structures and the sequence according to which they are used within the algorithm. The VND approach proposed by Henn (2015) performed quite well for the JOBASP. The author suggested utilization of the batch-related neighborhood structure $\mathcal{N}_1$ and the order-related structures $\mathcal{N}_3$ to $\mathcal{N}_6$. We also use these structures and the corresponding sequence. Furthermore, we introduce a new neighborhood structure $\mathcal{N}_2$ in order to be able to break up a complete batch and reinsert the orders into other batches. The impact of the moves applied according to the neighborhood structures is depicted in Table 4.

For problem classes with 5 order pickers and 200 customer orders, Table 4 includes information about the average proportion [in %] of the improvement obtained within the local search phases according to the neighborhood structure $\mathcal{N}_l$ ($l \in \{1, \ldots, 6\}$). For example, the entry 25.9 ($C = 45,\ \gamma = 0.6,\ l = 1$) means that 25.9% of the total improvement obtained by the VND can be attributed to the local search phases within the first neighborhood structure $\mathcal{N}_1$. The average proportion of the improvement ranges from 9.7% ($\mathcal{N}_5$) to 28.3% ($\mathcal{N}_2$). Therefore, it can be concluded that all neighborhood structures should be integrated into the algorithm.

The batch-related neighborhood structure $\mathcal{N}_1$ is responsible for 24.2% of the total improvement. It can be observed that the impact of this structure is much higher than the impact of the order-related neighborhood structures $\mathcal{N}_3$–$\mathcal{N}_6$, whose proportion of the improvement amounts to approximately 10%, respectively. This can be traced back to the fact that the batch-related neighborhood structure is the first structure of the sequence and applied much more often than order-related structures. Nevertheless, the impact of structures $\mathcal{N}_3$–$\mathcal{N}_6$ should not be underestimated as improvements found in the respective local search phases may allow the algorithm to further improve the solution by continuing with $\mathcal{N}_1$ and $\mathcal{N}_2$.

The newly proposed neighborhood structure $\mathcal{N}_2$ shows the largest proportion (28.3%) of the total improvement, although it is sequenced after $\mathcal{N}_1$. The proportion strongly fluctuates, ranging from 2.1% ($C = 45,\ \gamma = 0.8$) to 76.0% ($C = 75,\ \gamma = 0.6$). The reason can be found in the generation of the initial solution. As shown in the previous subsection, the ESDR-based algorithm leads to good solutions for loose due dates which are easy to meet (instances with a low MTCR γ or a medium MTCR and a large capacity C), while it is outperformed by the seed algorithm when the due dates get tight. A move according to $\mathcal{N}_2$ is defined by breaking up a complete batch and reassigning the orders to other batches. If the initial solution is constructed by means of the seed algorithm, these moves will usually not lead to improvements since the orders are already grouped into batches in a reasonable manner which also takes into account the processing times. This is not true for solutions generated by applying the ESDR-based algorithm. In this approach, batches are constructed considering the due dates of the orders only, which is the reason why the impact of $\mathcal{N}_1$ (which simply changes the position of batches) is quite small. However, by means of moves according to $\mathcal{N}_2$, the batching can be optimized regarding the processing times resulting in massive improvements with respect to the tardiness of all orders.

6.3. Evaluation of the performance of the VND approach

6.3.1. The VND and an exact approach applied to small and medium-sized problem instances

In order to evaluate the solution quality of the proposed VND algorithm, the objective function values of the solutions provided by this approach could be compared to the ones of an exact solution approach. For this purpose, we generated small ($n \in \{10, 20\}$ orders) and medium-sized ($n = 50$ orders) instances with $p = 2$ or $p = 5$ order pickers. For each problem class, 10 instances have been generated and solved by means of the newly-proposed model formulation. Application of the IP-solver has been terminated after 7200 seconds. Problematic is the fact that solving the LP-relaxation of the model almost always results in a total tardiness equal to 0. Also, no other procedures are known for the determination of adequate lower bounds for the JOBASRP (Henn, 2015). Consequently, optimality gaps cannot be obtained. For an evaluation of the performance of the VND algorithm, we considered instances with MTCRs of $\gamma = 0.6$ and $\gamma = 0.8$ and compared the respective VND solution to the best solution found by the IP-solver (applied to the model) within 2 hours of computing time.

In Table 5, the average total tardiness per problem instance is presented for solutions generated by the IP-solver as well as for solutions provided by the VND algorithm. Furthermore, w.r.t. the VND algorithm, the average improvement (imp) [in %] of the objective function value (tardiness) in comparison to the best objective function value obtained by the IP-solver is given. Finally, average computing times are depicted for both approaches. The results demonstrate that the VND algorithm outperforms the exact approach by far, both in terms of solution quality and computing time. What concerns small instances with $n = 10$, the exact approach generated solutions of acceptable quality. In fact, for a low MTCR ($\gamma = 0.6$), three instances have been solved to optimality (two instances for $p = 2$ and one instance for $p = 5$). For such small problem instances, though, assignment and sequencing decisions are rather trivial since the number of batches is not larger than the number of pickers. Consequently, the total tardi-

Table 5
Results for the exact approach and for the VND algorithm.

n	p	γ	**Exact approach**		**VND**		
			time	tar	time	tar	imp
10	2	0.6	5822	9	2	8	8.7
10	2	0.8	7200	75	3	70	6.7
10	5	0.6	6600	26	3	23	11.4
10	5	0.8	7200	77	3	74	4.2
20	2	0.6	7200	67	8	25	63.3
20	2	0.8	7200	307	14	220	28.5
20	5	0.6	7200	63	14	34	45.6
20	5	0.8	7200	206	12	166	18.9
50	2	0.6	7200	17,249	31	34	99.7
50	2	0.8	7200	20,342	98	1167	93.6
50	5	0.6	7200	6094	33	52	99.0
50	5	0.8	7200	7161	85	622	90.6

ness is minimized by assigning at most one order to each picker. Nevertheless, even for these small instances, the exact approach was outperformed by the VND algorithm. The latter did not only manage to find optimal solutions for the three instances which have been solved optimally by the exact approach; in addition to that it provided solutions with a smaller average total tardiness for all four problem classes, where the improvements ranged between 4.2% and 11.4%. When the number of customer orders is increased, it can be observed that the solution quality of the exact algorithm drastically deteriorates. For medium-sized instances ($n = 50$), the exact approach produced solutions of inferior quality and application of the VND algorithm led to improvements ranging between 90.6% ($p = 5,\ \gamma = 0.8$) and 99.7% ($p = 2,\ \gamma = 0.6$).

We conclude that the discussed exact solution procedure, i.e. applying a commercial IP-solver to the respective model formulation, does by no means represent a competitive or even promising approach to the JOBASRP. Even for very small problem instances, the exact approach is outperformed by the proposed VND algorithm.

6.3.2. Generation of upper bounds for large problem instances

Since we will not be able to evaluate the solution quality of the proposed VND algorithm by comparing the objective function values of the provided solutions to the respective optimal objective function values or to an adequate lower bound, we will have to use upper bounds from the application of heuristic approaches as a reference instead. Three approaches will be used for generating upper bounds.

In the first approach, solutions are provided by the ESDR-based algorithm in combination with the S-shape routing strategy. The ESDR-based algorithm, which has also been used by Henn (2015), is a very straightforward approach to solve the JOBASRP. Solutions from this relatively simple approach and the corresponding objective function values will be used in order to identify what the benefits are when turning to a more complex approach (i.e. to our VND algorithm).

In practice, the respective subproblems of the JOBASRP are solved in sequence. We, therefore, designed a sequential solution approach in which state-of-the-art algorithms have been integrated for solving these subproblems. The results from this approach will be compared to those from the VND algorithm in order to identify the benefits from dealing with the JOBASRP as a holistic problem. In the first step of the sequential approach, each order is contained in a single batch; the corresponding processing times are determined on the basis of the S-shape strategy. Then, the BASP is to be solved, which results in an assignment of customer orders to order pickers. As mentioned in Section 3, when each batch includes one order only, the BASP is equivalent to the Parallel Machine Scheduling Problem with the objective of minimizing the total tardiness of all orders. We have implemented the solution approach of Biskup, Herrmann, and Gupta (2008) here which currently represents the state-of-the-art algorithm for this problem (Ullrich, 2013). In the second step, given the previously-generated assignment, for each order picker, the JOBASP is solved by the iterated local search (ILS) algorithm of Henn and Schmid (2013). The ILS algorithm is terminated after a certain time limit has been reached. The time limit t^{ILS} is chosen in such a way that the computing time of the sequential approach coincides with the time t^{VND} needed for applying the VND algorithm. Since solving a single problem instances requires for applying the ILS algorithm to the JOBASP of each order picker, the time limit of the ILS is set to $t^{\text{ILS}} = t^{\text{VND}}/p$, where p denotes the number of pickers. Finally, in the third step, in order to reduce the processing times of the batches, the LKH heuristic is applied to each of the routing problems arising from the customer orders included in the batches.

As a third upper bound, we use the initial solution to which the VND in combination with the LKH heuristic is applied. This bound is chosen in order to evaluate the improvements which are obtained by application of the VND algorithm.

6.3.3. Solution quality of the VND algorithm for large instances

In Tables 6 and 7, the average total tardiness (tar) in minutes is depicted for the ESDR-based algorithm combined with S-shape routing (ESD), the sequential solution approach (SEQ), the initial solution after the application of the LKH heuristic (INI) and the complete variable neighborhood descent algorithm (VND) for problem classes with 100 and 200 orders, respectively. Furthermore, the average improvements (imp_i) [in %] are presented compared to the total tardiness provided by approach i ($i \in \{1, 2, 3\}$) from the previous subsection.

Comparison with the ESDR-based algorithm: When comparing solutions obtained by the ESDR-based algorithm with VND solutions, significant improvements regarding the total tardiness of all customers can be observed. On average, the reduction of the total tardiness ranges from 51.5% (200 orders, 2 pickers, $C = 75,\ \gamma = 0.6,\ q = 1$) to 95.2% (200 orders, 3 pickers, $C = 45,\ \gamma = 0.7,\ q = 2$). The number of pickers does not seem to have a strong impact on the amount of improvement as the average reduction is between 69.2% and 71.2% for 100 customer orders and ranges from 73.5% to 74.8% for 200 orders. However, the impact of the other factors under investigation, namely the number of orders, the capacity C of the picking device, the MTCR γ as well as the number of blocks q does not seem to be negligible.

The more orders have to be assigned to batches, the larger the number of solutions gets since more combinations of orders exist which can be grouped into a batch. In the ESDR-based algorithm, orders are sequentially assigned to batches not taking advantage of the larger number of possibilities. In contrast to this, the VND considers much more moves according to the neighborhood structures $\mathcal{N}_3$ to $\mathcal{N}_6$. Thus, it is not surprising that the amount of improvement increases with an increasing number of customer orders. An increasing capacity of the picking device leads to a reduction with respect to the relative improvement. A larger capacity leads to tours containing more pick locations. Since the S-shape strategy is known to perform quite well if the number of pick locations is large in comparison to the number of subaisles (Roodbergen, 2001), the difference between the S-shape strategy and the LKH heuristic in terms of solution quality is quite small in these cases, resulting in less space for improvement. The reverse line of argumentation holds for the impact of an increase in the number of blocks as more blocks result in more subaisles, leading to a smaller number of pick locations per subaisle. Thus,

Table 6
Evaluation of the VND for 100 orders.

C	γ	q	2 pickers							3 pickers							5 pickers						
			ESD	**SEQ**	**INI**	**VND**				**ESD**	**SEQ**	**INI**	**VND**				**ESD**	**SEQ**	**INI**	**VND**			
			tar	tar	tar	tar	imp_1	imp_2	imp_3	tar	tar	tar	tar	imp_1	imp_2	imp_3	tar	tar	tar	tar	imp_1	imp_2	imp_3
45	0.6	1	181	141	98	54	70.1	61.6	45.1	153	137	86	46	69.8	66.2	46.4	121	136	73	43	64.6	68.6	41.9
45	0.6	2	180	186	55	36	80.0	80.7	34.7	340	230	122	55	83.7	75.9	54.6	281	216	100	54	80.9	75.1	46.3
45	0.6	3	525	204	107	65	87.5	67.9	39.1	716	318	156	87	87.8	72.6	44.2	1052	415	236	148	85.9	64.3	37.2
45	0.7	1	2754	909	840	359	87.0	60.5	57.3	2061	743	906	312	84.9	58.0	65.6	1621	712	824	323	80.0	54.5	60.8
45	0.7	2	4647	1494	1140	545	88.3	63.5	52.2	3712	1303	1452	665	82.1	49.0	54.2	2415	1012	1099	451	81.3	55.4	58.9
45	0.7	3	7751	2336	1606	995	87.2	57.4	38.1	5544	1598	1605	741	86.6	53.6	53.8	3886	1414	1535	749	80.7	47.0	51.2
45	0.8	1	9145	4586	4614	3590	60.7	21.7	22.2	6880	3439	3723	2826	58.9	17.8	24.1	4301	2322	2526	1873	56.4	19.3	25.8
45	0.8	2	12,773	6293	6076	4859	62.0	22.8	20.0	8730	4136	4422	3345	61.7	19.1	24.3	5705	2978	3107	2362	58.6	20.7	24.0
45	0.8	3	16,987	7628	7123	5750	66.1	24.6	19.3	11,980	5473	5363	4306	64.1	21.3	19.7	7780	3674	3716	2892	62.8	21.3	22.2
75	0.6	1	71	63	57	33	53.2	47.2	42.1	96	62	77	35	63.0	43.0	54.2	130	81	107	35	73.3	56.7	67.5
75	0.6	2	81	61	58	30	62.5	50.1	47.8	122	98	90	42	65.4	57.1	53.3	167	109	118	46	72.5	57.8	60.9
75	0.6	3	119	78	65	41	65.6	47.8	37.4	203	118	100	53	74.0	55.2	47.4	350	219	193	109	68.9	50.3	43.6
75	0.7	1	174	140	127	83	52.4	40.9	34.5	211	195	142	84	60.3	56.9	40.8	319	294	235	143	55.3	51.4	39.4
75	0.7	2	289	220	153	105	63.8	52.4	31.3	528	362	279	171	67.7	52.7	38.6	462	381	270	161	65.1	57.6	40.2
75	0.7	3	750	365	205	154	79.5	57.9	24.9	798	423	268	187	76.5	55.8	30.1	1083	551	377	257	76.3	53.4	32.0
75	0.8	1	3397	1537	2204	1170	65.6	23.9	46.9	2969	1567	2206	1211	59.2	22.7	45.1	2193	1321	1672	931	57.6	29.6	44.3
75	0.8	2	5319	2366	3066	1737	67.3	26.6	43.3	3961	1835	2446	1315	66.8	28.3	46.2	3046	1727	2077	1200	60.6	30.5	42.3
75	0.8	3	8238	3160	3792	2294	72.2	27.4	39.5	6243	2602	3333	1972	68.4	24.2	40.8	4487	2149	2558	1563	65.2	27.2	38.9
Average			4077	1765	1744	1217	70.6	46.4	37.5	3069	1369	1488	970	71.2	46.1	43.5	2189	1095	1157	741	69.2	46.7	43.2

Table 7
Evaluation of the VND for 200 orders.

C	γ	q	2 pickers							3 pickers							5 pickers						
			ESD	**SEQ**	**INI**	**VND**				**ESD**	**SEQ**	**INI**	**VND**				**ESD**	**SEQ**	**INI**	**VND**			
			tar	tar	tar	tar	imp_1	imp_2	imp_3	tar	tar	tar	tar	imp_1	imp_2	imp_3	tar	tar	tar	tar	imp_1	imp_2	imp_3
45	0.6	1	228	151	114	63	72.6	58.6	45.2	266	241	141	63	76.3	73.9	55.4	253	245	143	76	69.8	68.9	46.5
45	0.6	2	295	177	99	53	82.2	70.2	46.7	284	363	91	58	79.5	84.0	36.4	591	447	203	98	83.5	78.1	51.8
45	0.6	3	1978	497	232	97	95.1	80.5	58.1	1008	513	135	94	90.7	81.7	30.5	1026	629	199	130	87.3	79.3	34.6
45	0.7	1	10,219	2166	2304	671	93.4	69.0	70.9	7816	1892	2959	552	92.9	70.8	81.3	4745	1418	2224	433	90.9	69.5	80.5
45	0.7	2	16,722	3715	3038	1040	93.8	72.0	65.8	11,392	2504	3475	551	95.2	78.0	84.2	7071	1905	2491	505	92.9	73.5	79.7
45	0.7	3	28,590	5761	3980	1815	93.7	68.5	54.4	19,366	4323	5192	1455	92.5	66.3	72.0	13,585	3686	5052	1475	89.1	60.0	70.8
45	0.8	1	37,946	17,689	18,075	14,069	62.9	20.5	22.2	25,665	11,994	13,208	9535	62.8	20.5	27.8	16,298	7765	8966	6311	61.3	18.7	29.6
45	0.8	2	50,187	22,165	22,667	17,632	64.9	20.5	22.2	32,649	14,481	15,722	11,283	65.4	22.1	28.2	21,624	9855	11,260	7912	63.4	19.7	29.7
45	0.8	3	66,036	27,281	26,109	20,557	68.9	24.6	21.3	45,702	19,235	19,245	14,804	67.6	23.0	23.1	28,415	13,570	13,058	9511	66.5	29.9	27.2
75	0.6	1	78	61	63	37	51.9	38.8	40.5	98	71	78	37	62.7	48.2	53.0	156	125	123	54	65.1	56.5	55.8
75	0.6	2	90	60	63	33	62.9	45.0	47.1	123	95	89	43	65.2	54.7	51.6	213	175	148	71	66.8	59.6	52.1
75	0.6	3	139	91	63	36	73.7	59.9	42.0	204	138	114	68	66.7	50.7	40.7	340	233	196	103	69.7	55.8	47.6
75	0.7	1	229	235	146	99	56.6	57.8	31.9	242	249	163	93	61.7	62.9	43.3	316	350	236	137	56.7	60.9	42.1
75	0.7	2	560	478	326	218	61.1	54.5	33.3	361	286	193	126	65.0	55.8	34.6	437	379	246	135	69.0	64.3	44.9
75	0.7	3	924	406	221	160	82.7	60.7	27.7	1143	615	317	221	80.6	64.0	30.3	1414	810	393	264	81.3	67.4	32.7
75	0.8	1	13,381	5090	8172	3687	72.4	27.6	54.9	9484	4030	6704	2804	70.4	30.4	58.2	6763	3331	5092	2280	66.3	31.5	55.2
75	0.8	2	20,149	7281	10,792	5165	74.4	29.1	52.1	13,026	4963	7953	3148	75.8	36.6	60.4	9832	4351	6375	2913	70.4	33.1	54.3
75	0.8	3	29,080	9331	12,385	6127	78.9	34.3	50.5	21,485	7564	11,051	5156	76.0	31.8	53.3	14,462	5704	8279	3922	72.9	31.2	52.6
Average			15,379	5702	6047	3975	74.6	49.6	43.7	10573	4087	4824	2783	74.8	53.1	48.0	7086	3054	3593	2018	73.5	53.2	49.3

the solution quality of the S-shape policy deteriorates and the amount of the improvement obtained by applying the VND (into which two more sophisticated routing algorithms are integrated) increases. The MTCR determines how tight the due dates are. A larger MTCR leads to tighter due dates which tend to increase the total tardiness of solutions to the corresponding problem instance. Apart from problem classes with an MTCR $\gamma = 0.8$ and a capacity $C = 45$, the amount of improvement increases with an increasing MTCR. This can be explained by the fact that the solution quality of the ESDR-based algorithm deteriorates when the due dates get tighter (see Section 6.2.1). If the MTCR is low or medium ($\gamma \in \{0.6, 0.7\}$), the VND leads to solutions with an average total tardiness of up to 1815 minutes (200 orders, 2 pickers, $C = 45$, $\gamma = 0.7$, $q = 3$), i.e. the average tardiness of an order amounts to 9 minutes. For instances of these problem classes, the VND provides many solutions with a total tardiness which is zero or close to zero, resulting in improvements of 100% (regardless of the absolute amount of improvement). Thus, conclusions have to be drawn carefully for instances with a small MTCR. If the MTCR is large ($\gamma = 0.8$), the average total tardiness significantly increases. Finding solutions with a low total tardiness gets even harder when the capacity is small ($C = 45$) since, the total processing times of the orders increase. Therefore, the amount of the relative improvement decreases when very difficult instances are considered. Nevertheless, the VND manages to massively reduce the total tardiness as an absolute improvement of up to 45,479 minutes (200 orders, 2 pickers, $C = 45$, $\gamma = 0.8$, $q = 3$) is achieved, which corresponds to a decrease of the tardiness by approximately 4 hours per order.

Comparison with the sequential solution approach: In the VND algorithm, neighborhood structures deal with the assignment and sequencing problem, and with the batching problem, while routing algorithms are integrated for the determination of processing times and for the evaluation of solutions. Thus, all three subproblems are considered jointly in the VND approach. The resulting holistic problem is very complex and difficult to solve. Therefore, we evaluate the benefit from using the integrated approach compared to solving the subproblems in sequence by applying a state-of-the-art algorithm to each subproblem, which is done in the second approach for generating upper bounds.

The results of the numerical experiments indicate that solving the subproblems jointly reduces the total tardiness by 49.2% on average across all problem classes. This clearly demonstrates that solving these subproblems simultaneously is pivotal for achieving high-quality solutions. The number of order pickers available for processing customer orders as well as the number of blocks in the picking area do not seem to have a large impact on the amount of improvement. The impact of the number of customer orders is also quite small. For 100 customer orders, the joint solution approach is able to decrease the total tardiness by 46.4%, while the relative improvement amounts to 52.0% on average for instances containing 200 orders. The largest impact on the solution quality of the sequential approach can be observed for the MTCR γ and the capacity C. If the MTRC is quite small ($\gamma = 0.6$), using the joint approach results in a reduction of the total tardiness by up to 84.0% (200 orders, 3 pickers, $C = 45$, $q = 2$). In this case, due dates of most customer orders can be met quite easily. This is verified by the VND algorithm which manages to find solutions, where the average tardiness of a customer order is less than a minute. Such high-quality solutions are rarely obtained when applying the sequential approach. This stems from the large objective function values of solutions generated at an early stage of the solution process. In the first step of the sequential approach, orders are assigned to order pickers and the simple S-shape strategy is used to determine processing times. The batching and the routing problems are not considered by the assignment algorithm at all. It only ensures that the sum of the processing times of the orders is fairly evenly distributed across all pickers. Consequently, nearly all customer orders get delayed significantly, where the average tardiness of an order amounts to more than 140 minutes. Based on the generated assignment, the algorithms applied for subsequent batching and routing are not capable of providing high-quality solutions to the joint problem. With an increasing MTCR, assignment decisions get less important as it is not possible to meet the due dates of the orders. When orders are tardy anyway, it is more pivotal to find good solutions to the batching and routing problems in order to decrease the processing times. Nevertheless, even for a large MTCR ($\gamma = 0.8$), the VND clearly outperforms the sequential approach, reducing the total tardiness by at least 17.8% (100 orders, 3 pickers, $C = 45$, $q = 1$).

Depending on the MTCR, an increasing capacity has different impacts on the amount of improvement. If the MTCR is small ($\gamma = 0.6$), increasing the capacity of the picking device implies less improvement by the VND. As pointed out before, in case of a small MTCR, the sequential approach leads to solutions of very poor quality because of bad assignment decisions. However, when the capacity of the picking device is large, the subsequent batching algorithm has lots of options to improve the solution, making the assignment decisions to have less impact on the solution quality. For a large MTCR ($\gamma = 0.8$), the relative improvement obtained by the VND increases with an increasing capacity. Whereas the impact is almost negligible for a small number of customer orders and order pickers, the difference gets very large when problems including 200 orders are considered. As mentioned before, finding high-quality solutions to the batching and routing problems for reducing processing times is pivotal in those instances, while the solution space gets quite large with an increasing number of orders and capacity of the picking device. Starting from a very poor solution, where each order is contained in a single batch, the ILS algorithm dealing with the batching subproblem is not able to reach the part of the search space containing very promising solutions within a small amount of computing time. (Note that the time limit for the ILS is smaller than the computing time required for applying the VND as the ILS has to be applied to the JOBASP of each picker.)

Comparison with the initial solution of the VND algorithm: In order to investigate the performance of the VND, we also compare the solutions to the initial solution in combination with the LKH heuristic. It can be observed that the VND manages to reduce the total tardiness by between 19.3% (100 orders, 2 pickers, $C = 45$, $\gamma = 0.8$, $q = 3$) and 84.2% (200 orders, 3 pickers, $C = 45$, $\gamma = 0.7$, $q = 2$). On average, across all problem classes, the improvement amounts to 44.2%, which demonstrates that the application of the VND is pivotal for the generation of high-quality solutions. Three main factors can be identified which have an impact on the amount of the improvement obtained. First, as observed in the comparison to the ESDR-based algorithm, a larger number of orders allows for more space for improvement. Second, the amount of improvement tends to decrease with an increasing number of blocks. Third, the combination of the MTCR γ and the capacity C has a great impact. While the largest improvements can be obtained for $\gamma = 0.7$ and $C = 45$, the smallest reductions are observed for $\gamma = 0.8$ and $C = 45$. This can be explained by the way how the initial solution is generated. The combination $\gamma = 0.8$ and $C = 45$ results in instances with very tight due dates which are very difficult to meet. In this case, the seed algorithm leads to very good solutions and, therefore, is chosen for the construction of initial solutions. Due to the high quality of these solutions, only small improvements can be obtained by application of the VND. In contrast, the combination $\gamma = 0.7$ and $C = 45$ leads to due dates not tight enough for the seed algorithm but too tight for the ESDR-based algorithm. Thus, the quality of the initial solution is quite

Table 8
Computing time (in seconds) required by the VND algorithm.

C	γ	q	100 orders			200 orders		
			Number of pickers			Number of pickers		
			2	3	5	2	3	5
45	0.6	1	77	80	96	222	349	448
45	0.6	2	52	70	68	164	229	501
45	0.6	3	48	72	88	256	252	474
45	0.7	1	293	377	416	1494	2553	3091
45	0.7	2	198	264	289	1130	2022	2680
45	0.7	3	160	252	268	1348	2597	3347
45	0.8	1	342	371	367	2263	2990	3621
45	0.8	2	216	260	259	1951	2264	2875
45	0.8	3	204	247	292	1918	2488	3155
75	0.6	1	122	126	151	231	255	334
75	0.6	2	46	49	57	101	121	184
75	0.6	3	50	59	72	106	141	218
75	0.7	1	184	164	243	406	449	518
75	0.7	2	72	104	100	185	209	260
75	0.7	3	85	103	120	209	231	388
75	0.8	1	545	685	596	2472	3075	3044
75	0.8	2	276	319	310	1629	1820	1940
75	0.8	3	338	431	376	1775	2027	2284
Average			184	224	232	992	1337	1631

poor and the VND manages to significantly reduce the total tardiness.

6.3.4. Considerations regarding computing times

Apart from the solution quality, the VND is evaluated with respect to the computing times required. Henn (2015) proposed solution approaches to the JOBASP using very simple routing strategies and reported computing times of up to 25 minutes for problem instances with 200 orders. It can be expected that our solution approach requires a higher computational effort as we integrated more complex routing algorithms. Furthermore, Henn (2015) considered a single-block layout only. Especially when the capacity is large and the simple S-shape strategy is used, many batches will result in the same tours as all picking aisles will be traversed. Due to this characteristic, the problem is reduced to a sequencing problem and solution approaches will terminate much faster as there is less room for improvement. This is not true for our setup since we also consider more complex layouts. Furthermore, tours constructed by the LKH heuristic are dependent on the certain pick locations instead of the subaisles to be visited only as it is the case for the S-shape strategy.

In Table 8, the average computing time required by the VND for solving a problem instance is depicted for all problem classes. As can be seen, computing times are dependent on the number of customer orders as well as on how difficult the due dates can be met. Of course, a larger number of orders significantly increases the computing times as it allows for much more possible moves regarding the neighborhood structures. For problem classes with 100 orders, the computing times are not a critical issue as they range from 1 to 11 minutes. When 200 orders are considered, however, up to 1 hour is required for solving a single instance. The largest computing times can be observed for problems with a large MTCR ($\gamma = 0.8$) and a small capacity ($C = 45$), which can be explained by the fact that these instances are characterized by very tight due dates. In practical applications, such high computing times may be a critical issue, which is why we investigate to which extent the solution quality decreases if less computing time is spent. Since the largest computing times arise for instances with

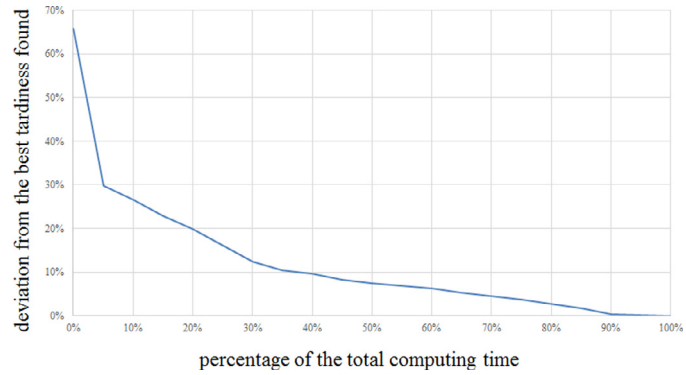

Fig. 7. Development of the solution quality over time.

200 orders, 5 pickers, $C = 45$, $\gamma = 0.8$ and $q = 1$, we depicted the development of the solution quality over time for these instances in Fig. 7.

The VND starts with an initial solution in which the combined heuristic is used in order to determine the processing times of the batches. The computing time for constructing this solution is negligible. Then, the LKH heuristic is applied, significantly reducing the processing times and improving the objective function value. On average, application of the LKH heuristic to all batches only requires 12 seconds of computing time for instances from this problem class. The remaining improvement is obtained by the moves corresponding to the different neighborhood structures. As can be seen in Fig. 7, the largest proportion of the improvement is realized within the first 30% of the computing time. After investing 40% of the time, which corresponds to 24 minutes, the total tardiness is 10% above the tardiness of the best solution found after the VND terminates. However, it can be observed that the VND manages to steadily improve the solution until the end of the solution process, i.e. reducing the computing time will definitely result in a larger total tardiness.

If the solution process has to be speeded up, the removal of $\mathcal{N}_6$ would be a straightforward way as this neighborhood structure requires the highest computational effort. Considering instances from the problem class defined above, a removal of $\mathcal{N}_6$ would save 44.0% of the total computing time, while the average total tardiness would increase by 6.4%. Another possible modification for saving computing time consists in the application of other routing algorithms. In order to determine the processing times of the batches, which is necessary to evaluate a solution, a routing problem has to be solved for each batch. This is done by means of the combined heuristic which represents a very fast solution approach. However, whenever a local optimum has been found, the processing times of the batches are improved by applying the LKH heuristic. This step consumes a large proportion of the total computing time required by the VND. Therefore, the impact of integrating routing algorithms, which are based on more simple improvement strategies (e.g. 2-opt or 3-opt), is evaluated in Table 9.

Table 9 depicts the relative average deviation of the total tardiness (Δtar) [in %] as well as the deviation of the computing time (Δtime) [in %] when applying the 2-opt or 3-opt improvement strategies to a local optimum instead of using the LKH heuristic. Only problem classes with 200 orders, 5 pickers, an MTCR γ equal to 0.8 or with $\gamma = 0.8$ and a capacity C of 45 are considered here, as instances from these classes clearly require the highest amount of computing time when being solved by means of the VND algorithm. As can be seen from the table, the tardiness increases while computing times decrease when using more simple improvement strategies. On average, over all problem classes under consideration, the tardiness increases by 15.6% (2-opt) and 5.0% (3-opt),

Table 9
Results for the application of the 2-opt and 3-opt improvement strategies instead of the LKH heuristic.

C	γ	q	**LKH**		**2-opt**				**3-opt**			
			tar	time	tar	Δtar	time	Δtime	tar	Δtar	time	Δtime
45	0,7	1	433	3091	555	28.1	2153	−30.3	460	6.1	2247	−27.3
45	0,7	2	505	2680	676	33.8	2068	−22.8	548	8.5	2084	−22.2
45	0,7	3	1475	3347	1735	17.6	2740	−18.1	1553	5.3	2951	−11.8
45	0,8	1	6311	3621	6821	8.1	2270	−37.3	6497	2,9	2456	−32.2
45	0,8	2	7912	2875	8519	7.7	2277	−20.8	8135	2,8	2078	−17.3
45	0,8	3	9511	3155	10,141	6,6	2536	−19.6	9728	2.3	2624	−16.8
75	0,8	1	2280	3044	2525	10.7	1618	−46.8	2373	4.1	2140	−29.7
75	0,8	2	2913	1840	3368	15.6	1225	−33.4	3114	6.9	1541	−16.2
75	0,8	3	3922	2284	4382	11.7	1783	−21.9	4161	6.1	1934	−15.3
Average			3918	2882	4302	15.6	2075	−27.9	4063	5.0	2228	−21.0

whereas the average computing time is reduced by 27.9% (2-opt) and 21.0% (3-opt). Thus, the results indicate that using more simple improvement strategies is a valid option if computing times are a critical issue. This holds for the 3-opt improvement strategy in particular. Whereas the integration of the 2-opt strategy leads to an increase of the total tardiness by up to 33.8% ($C = 45$, $\gamma = 0.7$, $q = 2$), using 3-opt results in an acceptable solution quality in all problem classes. Considering the problem class where solving instances by means of the original VND algorithm consumes the largest amount of computing time ($C = 45$, $\gamma = 0.8$, $q = 1$), applying 3-opt instead of the LKH heuristic reduces the average computing time by 32.2%, while increasing the average total tardiness by 2.9% only. Thus, we conclude that the VND can be adjusted, if necessary, in such a way that solutions with a reasonable quality can be found within an acceptable amount of computing time.

7. Conclusion

In this paper, we considered the Joint Order Batching, Assignment and Sequencing, and Routing Problem which is rarely addressed in the literature, although it is pivotal for an efficient organization of manual order picking systems. We proposed a new mathematical model formulation. In contrast to existing formulations, the size of the model increases polynomially with the number of customer orders, which allows for solving small instances within a reasonable amount of computing time. Furthermore, we designed a variable neighborhood descent approach which is able to deal with very large problems. In order to reach the local optimum quite fast and speed up the solution process, a new heuristic for the construction of an initial solution is developed which outperforms the earliest start date rule-based algorithm by far when due dates are very tight.

By means of numerical experiments, the solution quality of the variable neighborhood descent algorithm is evaluated. First, the initial solution is compared to solutions obtained by applying the earliest start date rule-based algorithm, which represents a common approach to generate a solution and was also used by Henn (2015). It is demonstrated that our initial solution leads to a reduction of the total tardiness by up to 63%. In a second step, we show that all neighborhood structures are important in order to obtain high-quality solutions. The largest proportion of the total improvement is achieved by moves regarding a newly designed neighborhood structure which breaks up complete batches and reassigns the orders to other batches. Finally, several approaches for generating upper bounds are used in order to evaluate the quality of solutions provided by the variable neighborhood descent algorithm. Combining the earliest start date rule-based algorithm with the simple S-shape strategy represents a very straightforward way to solve the problem. However, it is pointed out that the solution quality of this approach is very poor as the variable neighborhood descent manages to improve the objective function value of these solutions by up to 95%, which means a massive reduction of the total tardiness. Another upper bound is obtained by decomposing the Joint Order Batching, Assignment and Sequencing, and Routing Problem into its subproblems and sequentially solving them by means of state-of-the-art algorithms. Thus, this upper bound represents the total tardiness which results from considering the subproblems separately as done in the literature so far. Compared to this sequential solution approach, applying the variable neighborhood descent algorithm reduces the total tardiness of all customer orders by up to 84%, which demonstrates that dealing with the Joint Order Batching, Assignment and Sequencing, and Routing Problem as a holistic problem is inevitable for an efficient organization of warehouse operations.

We dealt with the Joint Order Batching, Assignment and Sequencing, and Routing Problem in warehouses with wide aisles which allow order pickers to overtake each other. Further research could concentrate on picker blocking aspects arising in narrow-aisle warehouses, where subaisles may be blocked when two pickers are moving in opposite directions or traffic jams may occur when several pickers have to visit the same storage location at the same time. When taking blocking aspects into account, tours of different pickers cannot be independently determined anymore, making the problem much more difficult to deal with. From a practical point of view, the on-line variant of the Joint Order Batching, Assignment and Sequencing, and Routing Problem would also be a very interesting topic as customer orders arrive during the day and are not known in advance, which requires for very fast solution approaches in order to be able to recalculate each time a certain number of new orders has arrived.

References

Biskup, D., Herrmann, J., & Gupta, J. N. D. (2008). Scheduling identical parallel machines to minimize total tardiness. *International Journal of Production Economics, 115*, 134–142.

Bozer, Y. A., & Kile, J. W. (2008). Order batching in walk-and-pick order picking systems. *International Journal of Production Research, 46*, 1887–1909.

Chen, T.-L., Cheng, C.-Y., Chen, Y.-Y., & Chan, L.-K. (2015). An efficient hybrid algorithm for integrated order batching, sequencing and routing problem. *International Journal of Production Economics, 159*, 158–167.

Elsayed, E., & Lee, M. K. (1996). Order processing in automated storage/retrieval systems with due dates. *IIE Transactions, 28*, 567–577.

Elsayed, E., Lee, M. K., Kim, S., & Scherer, E. (1993). Sequencing and batching procedures for minimizing earliness and tardiness penalty for order retrievals. *International Journal of Production Research, 31*, 727–738.

Elsayed, E. A. (1981). Algorithms for optimal material handling in automatic warehousing systems. *International Journal of Production Research, 19*, 525–535.

Gademann, N., van den Berg, J., & van der Hoff, H. (2001). An order batching algorithm for wave picking in a parallel-aisle warehouse. *IIE Transactions, 33*, 385–398.

Grosse, E. H., Glock, C. H., & Ballester-Ripoll, R. (2014). A simulated annealing approach for the joint order batching and order picker routing problem with weight restrictions. *International Journal of Operations and Quantitative Management, 20*, 65–83.

Hansen, P., & Mladenović, N. (2001). Variable neighborhood search: Principles and applications. *European Journal of Operational Research, 130*, 449–467.

Helsgaun, K. (2000). An effective implementation of the Lin–Kernighan traveling salesman heuristic. *European Journal of Operational Research, 126*, 106–130.

Henn, S. (2015). Order batching and sequencing for the minimization of the total tardiness in picker-to-part warehouses. *Flexible Services and Manufacturing, 27*, 86–114.

Henn, S., Koch, S., & Wäscher, G. (2012). Order batching in order picking warehouses: A survey of solution approaches. In R. Manzini (Ed.), *Warehousing in the global supply chain: Advanced models, tools and applications for storage systems* (pp. 105–137). London: Springer.

Henn, S., & Schmid, V. (2013). Metaheuristics for order batching and sequencing in manual order picking systems. *Computers & Industrial Engineering, 66*, 338–351.

de Koster, R., Le-Duc, T., & Roodbergen, K. J. (2007). Design and control of warehouse order picking: A literature review. *Science Direct, 182*, 481–501.

Koulamas, C. (1994). The total tardiness problem: Review and extensions. *Operations Research, 42*, 1025–1041.

Kulak, O., Sahin, Y., & Taner, M. E. (2012). Joint order batching and picker routing in single and multiple-cross-aisle warehouses using cluster-based tabu search algorithms. *Flexible Services and Manufacturing Journal, 24*, 52–80.

Letchford, A. N., Nasiri, S. D., & Theis, D. O. (2013). Compact formulations of the Steiner traveling salesman problem and related problems. *European Journal of Operational Research, 228*, 83–92.

Petersen, C. G., & Schmenner, R. W. (1999). An evaluation of routing and volume-based storage policies in an order picking operation. *Decision Science, 30*, 481–501.

Pinedo, M. L. (2016). *Scheduling: Theory, algorithms, and systems* (5th). Springer. Cham et al.

Ratliff, H. D., & Rosenthal, A. R. (1983). Order-picking in a rectangular warehouse: A solvable case of the traveling salesman problem. *Operations Research, 31*, 507–521.

Roodbergen, K. J. (2001). *Layout and routing methods for warehouses*. Trial: Rotterdam.

Roodbergen, K. J., & de Koster, R. (2001a). Routing methods for warehouses with multiple cross aisles. *International Journal of Production Research, 39*, 1865–1883.

Roodbergen, K. J., & de Koster, R. (2001b). Routing order pickers in a warehouse with a middle aisle. *European Journal of Operational Research, 133*, 32–43.

Scholz, A., Henn, S., Stuhlmann, M., & Wäscher, G. (2016). A new mathematical programming formulation for the single-picker routing problem. *European Journal of Operational Research, 253*, 68–84.

Scholz, A., & Wäscher, G. (2017). Order batching and picker routing in manual order picking systems: The benefits of integrated routing. *Central European Journal of Operations Research, 25*, 491–520.

Theys, C., Bräysy, O., Dullaert, W., & Raa, B. (2010). Using a TSP heuristic for routing order pickers in warehouses. *European Journal of Operational Research, 200*, 755–763.

Tompkins, J. A., White, J. A., Bozer, Y. A., & Tanchoco, J. M. A. (2010). *Facilities planning* (4th). New Jersey: John Wiley & Sons.

Tsai, C.-Y., Liou, J. J. H., & Huang, T.-M. (2008). Using a multiple-GA method to solve the batch picking problem: Considering travel distance and order due time. *International Journal of Production Research, 46*, 6533–6555.

Ullrich, C. (2013). Integrated machine scheduling and vehicle routing with time windows. *European Journal of Operational Research, 227*, 152–165.

Wäscher, G. (2004). Order picking: A survey of planning problems and methods. In H. Dyckhoff, R. Lackes, & J. Reese (Eds.), *Supply chain management and reverse logistics* (pp. 323–347). Berlin: Springer.